ALMA MATER STUDIORUM · UNIVERSITY OF BOLOGNA



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

School of Engineering

Department of Electrical, Electronics and Information Engineering "Guglielmo Marconi" (DEI)

Bachelor's Degree in Automation Engineering

Controlli Automatici T-2

Path Planning and Obstacle Avoidance for an Autonomous Agricultural Rover with Trailer: Trajectory Modification with Bézier Curves and Path Following Control

Author:
Tommaso Magelli

Supervisor:
Prof. Lorenzo Marconi

Co-Supervisor:
Ing. Nicolò Omodei

Academic year 2024 - 2025
Session II

ALMA MATER STUDIORUM · UNIVERSITY OF BOLOGNA



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

School of Engineering

Department of Electrical, Electronics and Information Engineering "Guglielmo Marconi" (DEI)

Bachelor's Degree in Automation Engineering

Controlli Automatici T-2

Path Planning and Obstacle Avoidance for an Autonomous Agricultural Rover with Trailer: Trajectory Modification with Bézier Curves and Path Following Control

Author:
Tommaso Magelli

Supervisor:
Prof. Lorenzo Marconi

Co-Supervisor:
Ing. Nicolò Omodei

Academic year 2024 - 2025
Session II

«I grandi amano le cifre. Quando voi gli parlate di un nuovo amico, mai si interessano alle cose essenziali. Non si domandano mai: "Qual è il tono della sua voce? Quali sono i suoi giochi preferiti? Fa collezione di farfalle?" Ma vi domandano: "Che età ha? Quanti fratelli? Quanto pesa? Quanto guadagna suo padre?"

Allora soltanto credono di conoscerlo.»

Antoine de Saint-Exupéry

Abstract

This thesis addresses the challenge of safe autonomous navigation for articulated agricultural rovers in unstructured open-field environments, a problem motivated by the development of an autonomous tractor at an agri-tech robotics startup. Standard path planning algorithms, while effective for rigid-body robots, generate kinematically unsafe trajectories for tractor-trailer systems due to the off-tracking phenomenon and their complex kinematics, leading to a risk of trailer collision with obstacles.

As a solution to this issue, this work presents an offline framework and a strategy for trajectory correction. The proposed system analyzes a reference path generated by a conventional planner and uses a preliminary simulation to gather data about the environment. A central contribution of this research is the development and comparative analysis of two hazard detection methods: a collision-based approach that uses a swept-path envelope to identify collisions, and a risk-based approach that combines a dynamic bounding circle with the vehicle's articulation angle to identify dangerous maneuvers.

Hazardous segments identified by these methods are then evaluated and replanned using Bézier curves, exploiting their mathematical properties to generate smooth, safe, and kinematically feasible alternatives. The framework's effectiveness is validated within a MATLAB simulation environment. Finally, the research delves into the execution phase with a comparative analysis of path-following controllers (Pure Pursuit and PID) to evaluate the trackability of the generated trajectories, examining how tuning influences the system's stability.

The results demonstrate that the proposed framework transforms unsafe initial paths into safe, drivable trajectories for a tractor and its trailer. This thesis provides a comparison of hazard detection philosophies and establishes a methodology for enhancing the operational safety of articulated autonomous systems in precision agriculture.

Contents

Abstract **vii**

Introduction **1**

1 Theoretical Background and State of the Art **5**

1.1 The Articulated Agricultural Rover: Kinematic Model and Navigation Challenges 5

1.1.1 Non-Holonomic Constraints and Configuration Space 7

1.1.2 Radius of Curvature of the Front Module and Off-tracking Phenomenon 8

1.1.3 System Equilibrium Angle and the Problem of Jackknifing 10

1.2 The Autonomous Navigation Pipeline: Perception, Planning, and Control . . . 11

1.3 Path Planning in Robotics: From Graph-Search to Sampling-Based Methods . . 13

1.3.1 Probabilistic Roadmaps (PRM) 14

1.4 Trajectory Tracking: A Review of Core Strategies 15

2 System Modeling and Path Generation Framework **19**

2.1 Development Environment: MATLAB and Robotics Toolbox 19

2.2 Modeling the Agricultural Rover and the Open-Field Environment 21

2.3 The Path Generation Workflow: From PRM to a Smoothed Path 23

2.4 Overview of the Offline Path Modification Framework 26

3 Development of the Trajectory Modification Framework **28**

3.1 Bézier Curves: The Mathematical Tool for Trajectory Replanning 28

3.2 Deterministic Hazard Detection Methodologies 30

3.3 The Maneuver Window: Causal Path Segmentation 32

3.4 Informed Replanning and Validation 34

4 Analysis and Discussion of Path Following Control **36**

4.1 Controller Implementation in the Simulation Environment 36

4.2 Performance Metrics for Path Tracking Evaluation 39

4.3 Experimental Results and Comparative Analysis 40

4.4 Parametric Analysis: The Influence of Lookahead Distance 42

5 Results and System Validation **44**

5.1 Case Study 1: Path Modification using the Envelope Approach (Method A) . . 46

5.2 Case Study 2: Path Modification using the Bounding Circle Approach (Method B) 48
5.3 Discussion and Comparison of Final Trajectories 52
5.4 Overall System Performance Evaluation 56

6 Conclusions and Future Work **58**
6.1 Summary of Work and Key Contributions 58
6.2 Limitations of the Current System 59
6.3 Avenues for Future Research and Development 59

Bibliography **61**

List of Figures **62**

Introduction

Context and Motivation: The Role of Automation in Modern Agriculture

In recent decades, scientific and technological progress has favored the development of a diverse range of mobile robots, resulting in a profound transformation in various industrial and service sectors. The use of autonomous vehicles is therefore becoming a topic of increasing interest across diverse fields. Many solutions are already being researched and employed for security surveillance operations, logistics, hazardous environment exploration, agriculture, and many more. These applications strongly leverage the advantages of unmanned robotics, including mitigated human risk exposure, increased operational precision, and the ability to perform repetitive, continuous tasks without fatigue.

Among these, precision agriculture – or agriculture 4.0 – has emerged as a particularly critical domain where robotic solutions look very promising in addressing growing global challenges. Among all, climate change and overpopulation are posing significant challenges to the agricultural sector, which needs to increase yields while minimizing environmental impact. This has led to the adoption of advanced technologies that enable precise resource allocation and data-driven decision-making, thereby enhancing both productivity and sustainability. Autonomous (Unmanned) Ground Vehicles (AGVs) have become key in this transformation, offering capabilities for crop monitoring, automated harvesting, and comprehensive field data collection, therefore driving the transition to "Smart Farming". These systems demonstrate immense potential to improve operational efficiency and sustainability, reduce labor costs, and enable 24/7 field operations.

Modern agricultural robots employ GPS, LiDAR, cameras, ultrasonic sensors, and inertial measurement units (IMU) that detect their environment through sophisticated sensor fusion, while performing complex navigation tasks in different settings. Autonomous systems have achieved sub-10cm trajectory tracking precision in recent developments, making them suitable for precision tasks such as seeding and spraying, as well as mechanical weeding operations that require precise odometry, position, and environmental data. The combination of machine learning, AI, and computer vision technologies has expanded their operational range to include real-time crop monitoring, yield prediction, and selective harvesting capabilities. Robotics is the ultimate vector to drive innovation in modern agriculture, enabling farmers to fuse their core knowledge with data-driven strategies both in managing and working on crops, fields, and vineyards.

Figure 1: Photo of the rover not included in this version.

Articulated Systems: Versatility and Navigational Complexity

Among the various robotic designs proposed by the scientific community, articulated mobile robots —commonly referred to as tractor-trailer systems- have garnered particular attention, both for their suitability in Agricultural environments and for their modularity and versatility, which, in many scenarios, makes them preferable to rigid-body robots when navigating in complex environments and uneven terrain. These robots are composed from two (or more) modules, which are modeled as identical as far as it regards mobility. These systems usually present a frontal, active module and a passively pulled module at the rear of the system.

The agricultural relevance of tractor-trailer systems extends beyond simple transportation, as they can enable autonomous tractors to trail classical agricultural implements such as seeders, sprayers, harvesters, and cultivation equipment, thus automating several processes. This modularity enables farmers to utilize a single tractor platform with multiple specialized trailers, optimizing both capital investment and operational flexibility. In addition, the weight distribution of articulated systems can reduce soil compaction compared to single heavy units, making these systems very apt for sustainable farming practices.

Despite these significant advantages and innovations, comprehensive research on articulated robotic systems remains significantly less common compared to studies and contributions on single-module robots, such as differential-drive or car-like vehicles. This research gap is due to the intrinsic increase in complexity of the control characteristics of tractor-trailer systems, rather than single body mobile robots, especially concerning path planning and obstacle avoidance in unstructured environments. This gap is even more pronounced in the agricultural robotics literature, where most autonomous navigation studies concentrate simply on autonomous tractors (rigid-body platforms) in structured environments, without taking into account the trailer dy-

namics and implications. The nonlinear and underactuated nature of these systems presents a clear challenge for control engineering to go beyond conventional control approaches and find new solutions.

The Open-Field Challenge: Path Planning and Obstacle Avoidance

This thesis dwells in the fundamental challenge of autonomous navigation in unstructured open-field environments, where effective obstacle avoidance and path evaluation is key for safe and reliable operation. Agricultural environments present a wide array of configurations that entail unique navigational challenges, including irregular terrain, dynamic obstacles such as livestock and farm workers, and the need to operate in close proximity to valuable crops and infrastructure such as vineyards or three rows.

The primary difficulty in controlling tractor-trailer systems is a kinematic phenomenon originating from the system's composition itself, and it's commonly referred to as off-tracking, where the trailer follows a significantly tighter turning radius than the tractor, creating increased collision risks with obstacles during turns and curved paths around dangerous areas. This phenomenon is particularly pronounced during low-speed maneuvers and sharp turns, which are characteristic of most of the more common agricultural operations, such as headland turns in row navigation and simple obstacle avoidance maneuvers. The magnitude of off-tracking can be substantial, with studies showing trailer path tracking errors of several meters in extreme cases.

Conventional path planning algorithms such as the Probabilistic Roadmap (PRM), Rapidly-exploring Random Trees (RRT), and all Dijkstra's derivations are inherently "naive" to the complex kinematics of articulated systems, as they track only the path to be followed. The scientific literature is replete with studies and enhancements on these algorithms, as they excel at finding collision-free paths by treating the robot as a simple geometric entity, but they fail to account for the dynamic coupling between the tractor and trailer modules. Consequently, "safe and feasible" paths calculated by these geometric planners may prove physically too dangerous or even impossible to execute without the trailer colliding with obstacles that were successfully avoided by the tractor. This limitation has been recognized as a significant barrier to the practical deployment of articulated autonomous vehicles in complex environments.

Recent research has highlighted the limitations of sampling-based planners when applied to articulated systems. Many of these planners, due to their very high computational cost, have the tendency to produce paths with narrow passages and sharp turns that heavily increase the risk of trailer collision and jackknifing configurations. The need for specialized path planning approaches that explicitly consider articulated vehicle kinematics has become necessary in the control of automated systems in agricultural automation.

Thesis Contribution and Structure

This thesis presents a comprehensive approach to path planning and obstacle avoidance specifically designed for autonomous agricultural rovers operating with trailers in unstructured field environments. The core contribution lies in the development of a novel off-line trajectory modification framework that identifies hazardous path segments generated by conventional geometric planners and replans them using advanced geometrical techniques to ensure both safety and kinematic feasibility. This work explores two fundamentally different hazardous segment identification protocols, comparing them and demonstrating a strategy to modify only critical segments of a trajectory, rather than the entire path.

The proposed system employs Bézier curves as the fundamental tool for trajectory modification, strongly leveraging their geometrical properties, including endpoint interpolation, convex hull containment, and differential continuity, which render these curves more apt for trajectory modification than splines or other parametric curves. As it will be seen later on, Bézier curves have particular utility in robotic path planning due to their ability to generate smooth, continuous, and differentiable trajectories that respect vehicle kinematic constraints while maintaining computational efficiency, limiting their presence in portions of the path that the planner deems as unfeasible. The framework incorporates non-real-time trajectory deformation capabilities in unstructured static environments, as it aims to create a modular, non-computationally expensive obstacle avoidance module, implementable with pre-planned paths or with classical geometric planners, enabling path modification only when a trailer is present. It's fair to mention that modern solutions to this problem utilize model predictive or receding horizon control (MPCs), exploring viable solutions to the backwards motion of tractor-trailer systems, but these topics are out of the scope of this thesis.

Beyond path planning and modification, this research extends to the final stage of the autonomous navigation pipeline by implementing and evaluating different trajectory tracking strategies. The investigation takes into account both classical and improved pure pursuit algorithms, analyzing their effectiveness in accurate trajectory tracking for articulated systems and comparing different tuning variables, and a simplistic PID controller, showing how careful tuning can meet the stringent accuracy requirements of precision farming operations.

The following chapters provide a detailed exposition of the theoretical foundations underlying articulated vehicle modeling and control, describe the development of a comprehensive simulation environment for system validation, present the novel hazard detection and trajectory modification methodologies, and analyze the comparative performance of various path following control approaches. The research concludes with a thorough discussion of experimental results, practical implementation considerations, and identification of promising avenues for future research in autonomous agricultural robotics.

Chapter 1

Theoretical Background and State of the Art

1.1 The Articulated Agricultural Rover: Kinematic Model and Navigation Challenges

The rover under consideration is an articulated mobile robot, composed of two distinct modules connected by a central pivot joint. The front module, or "tractor", is a differential drive unit equipped with tracks, providing all the propulsive force. The rear module, or "trailer", is passive and unpowered. Despite articulated robotic platforms widely varying in size, weight, and composition, a generalized kinematic model can be developed to describe the vehicle's motion accurately.

This similarity enables the development of a general kinematic model that can describe the motion of both modules.

A comprehensive analysis of these mobile robots can be approached from various perspectives, depending on the specific aspects of their behavior that need to be highlighted. Following Moreno's thorough analysis [1], two equivalent kinematic models are introduced to emphasize distinct characteristics of an articulated mobile robot. The kinematic model I'm going to describe defines the configuration space of the mobile robot, which encompasses the set of states necessary to describe the robot's configuration on the Euclidean plane (x_0, y_0).

The analysis of the first kinematic model is based on the following assumptions:

- The model is purely kinematic, neglecting dynamic forces.
- The model only considers movement on a planar surface; out-of-plane movements are disregarded.
- The locomotion modules are simplified by considering a single equivalent wheel for each side, with an axle passing through the module's reference point (O_1 or O_2).
- Each wheel is assumed to roll without slipping, meaning there is no longitudinal or transversal sliding.

Before proceeding with the kinematic study, a fixed planar reference system (x_0, y_0) is introduced, along with the angles φ_1 and φ_2, which represent the orientation of the front and rear modules, respectively, with respect to the x_0 axis (positive in the counter-clockwise direction)[1].

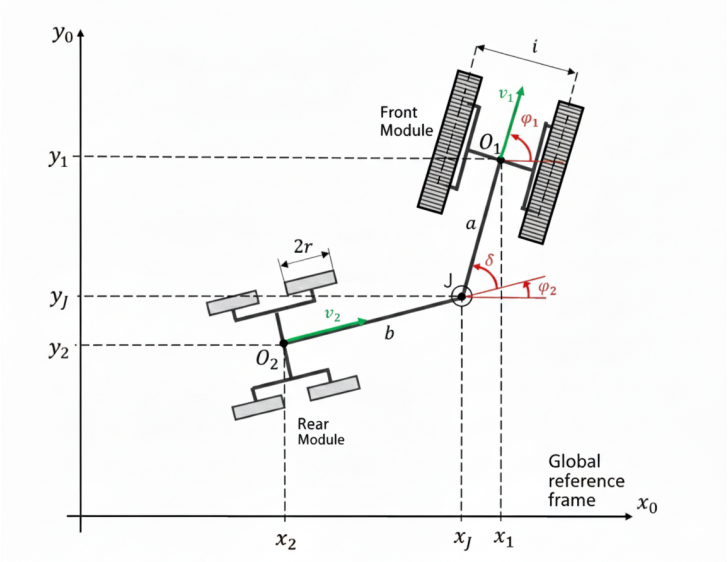


Figure 1.1: Simplified planar kinematic structure of the articulated robot. [1]

Figure 1.1 illustrates the simplified planar kinematic structure of the articulated robot. Each drive unit is in contact with the ground via a track. The position of the two modules, front and rear, is described by two frames. The origins of these frames (point O_1 for the front and point O_2 for the rear) are located at the intersection of the pivot axis of the drive unit carriers and the longitudinal axis. [1]

The pose of each of the two modules in the plane is determined by a set of three variables: two for the position and one for the orientation, as described in Table 1.1.

Table 1.1: Fundamental quantities of the robot.

Symbol	**Description**
(x_1, y_1, φ_1)	Pose of the front module (tractor)
(x_2, y_2, φ_2)	Pose of the rear module (trailer)
δ	Relative rotation (yaw) between the front and rear modules
a	Distance between the central joint and the front module
b	Distance between the central joint and the rear module
i	Transverse distance between the tracks
r	Radius of the wheels
v_1	Longitudinal velocity of the front module
$\dot{\varphi}_1$	Yaw rate of the front module
v_2	Longitudinal velocity of the rear module

[1]The z_0 axis, extending out of the (x_0, y_0) plane, defines the positive direction of rotation in the (x_0, y_0) plane.

The geometric relationships between the modules impose the following constraints:

$$x_2 = x_1 - b\cos\varphi_2 - a\cos\varphi_1 \tag{1.1}$$

$$y_2 = y_1 - b\sin\varphi_2 - a\sin\varphi_1 \tag{1.2}$$

$$\delta = \varphi_1 - \varphi_2 \tag{1.3}$$

1.1.1 Non-Holonomic Constraints and Configuration Space

Each track and wheel introduces a non-holonomic constraint on the system, as it does not allow for translation normal to the direction of rolling (it is not designed to slide sideways). For this reason, the normal velocity to the direction of rolling is null. Therefore, two additional non-holonomic constraints can be defined that limit the mobility of the moving vehicle:

$$\dot{x}_1\sin\varphi_1 - \dot{y}_1\cos\varphi_1 = 0 \tag{1.4}$$

$$\dot{x}_2\sin\varphi_2 - \dot{y}_2\cos\varphi_2 = 0 \tag{1.5}$$

The velocity in the global reference frame of each module is defined as:

$$\dot{x}_1 = v_1\cos\varphi_1 \tag{1.6}$$

$$\dot{y}_1 = v_1\sin\varphi_1 \tag{1.7}$$

$$\dot{x}_2 = v_2\cos\varphi_2 \tag{1.8}$$

$$\dot{y}_2 = v_2\sin\varphi_2 \tag{1.9}$$

The configuration space of the mobile robot, which is the set of states required to locate the robot's configuration in the plane, is expressed as:

$$q = \begin{bmatrix} x_1 & y_1 & \varphi_1 & \delta \end{bmatrix}^T \tag{1.10}$$

The kinematic model can be written as:

$$\dot{q} = A \begin{bmatrix} v_1 & \dot{\varphi}_1 \end{bmatrix}^T \tag{1.11}$$

The relationship that links $\dot{\delta}$ to the velocities v_1 and $\dot{\varphi}_1$ can be obtained by differentiating (1.1), (1.2), and (1.3) with respect to time and substituting the results into (1.5). It is thus possible to obtain:

$$\dot{\delta} = \left(\frac{a}{b}\cos\delta + 1\right)\dot{\varphi}_1 - \frac{1}{b}\sin\delta\cdot v_1 \tag{1.12}$$

Thus, the kinematic model can be written as:

$$\begin{bmatrix} \dot{x}_1 \\ \dot{y}_1 \\ \dot{\varphi}_1 \\ \dot{\delta} \end{bmatrix} = \begin{bmatrix} \cos\varphi_1 & 0 \\ \sin\varphi_1 & 0 \\ 0 & 1 \\ -\frac{1}{b}\sin\delta & \frac{a}{b}\cos\delta + 1 \end{bmatrix} \begin{bmatrix} v_1 \\ \dot{\varphi}_1 \end{bmatrix} \tag{1.13}$$

Assuming that the modules can be simplified by considering a single equivalent track for each side, it is possible to state that each module of the robot has differential steering with only two tracks. We will now proceed to determine the relationship that links the velocities v_{1l} and v_{1r}, respectively of the left and right tracks of the frontal module, to the longitudinal velocity v_1.

$$v_1 = \frac{1}{2}(v_{1r} + v_{1l}) \tag{1.14}$$

Furthermore, knowing the distance i between the tracks, the angular velocity $\dot{\varphi}_1$ is obtained:

$$\dot{\varphi}_1 = \frac{1}{i}(v_{1r} - v_{1l}) \tag{1.15}$$

Given the relationships (1.14) and (1.15), it is possible to rewrite the expression of the kinematic model in an alternative form:

$$\dot{q} = \begin{bmatrix} \frac{1}{2}\cos\varphi_1 & \frac{1}{2}\cos\varphi_1 \\ \frac{1}{2}\sin\varphi_1 & \frac{1}{2}\sin\varphi_1 \\ \frac{1}{i} & -\frac{1}{i} \\ \frac{a\cos\delta + b - \frac{i}{2}\sin\delta}{bi} & \frac{a\cos\delta + b + \frac{i}{2}\sin\delta}{bi} \end{bmatrix} \begin{bmatrix} v_{1r} \\ v_{1l} \end{bmatrix} \tag{1.16}$$

The velocities v_{1r} and v_{1l} can be related to the angular velocities of the wheels ω_{1r} and ω_{1l} by knowing the radius r of the wheel.

$$v_{1r} = r \cdot \omega_{1r} \tag{1.17}$$

$$v_{1l} = r \cdot \omega_{1l} \tag{1.18}$$

$$v_1 = \frac{r}{2}(\omega_{1r} + \omega_{1l}) \tag{1.19}$$

$$\dot{\varphi}_1 = \frac{r}{i}(\omega_{1r} - \omega_{1l}) \tag{1.20}$$

1.1.2 Radius of Curvature of the Front Module and Off-tracking Phenomenon

To describe a trajectory—whether straight, circular, or curvilinear—that the robot must follow, it is useful to introduce the concept of the radius of curvature. When studying two articulated rigid bodies, it must be considered that, in the most general case, they follow two distinct

trajectories during motion. This divergence between the paths of the front and rear modules is formally known as the off-tracking phenomenon. During any turn, the articulated linkage forces the rear module to follow a trajectory with a tighter radius of curvature than the front module, effectively causing it to "cut the corner." This behavior is a fundamental challenge in the control and navigation of articulated vehicles, as it can lead to unforeseen collisions if not properly modeled. This is equivalent to stating that each module is characterized by its own radius of curvature R and its own instantaneous center of rotation CIR.

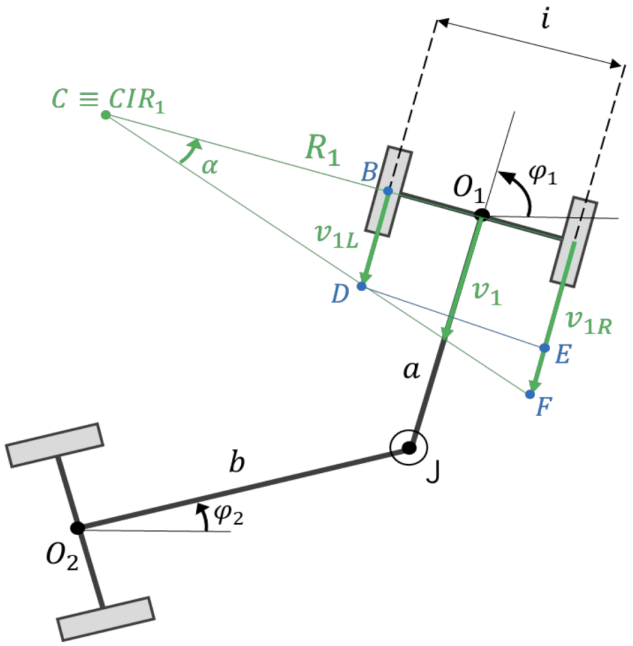


Figure 1.2: Representation of the turning radius of curvature of the front module. [1]

Since the front module is the only one equipped with drive tracks, the relationship that links the radius of curvature R_1 of the path followed by this module to the robot's kinematics is defined. We begin by determining the instantaneous center of rotation CIR_1: this is located at the intersection of the extension of the axle with the line connecting the velocity vectors of the two tracks.

The expression for the radius of curvature is obtained:

$$R_1 = i\left(\frac{1}{2} + \frac{v_{1l}}{v_{1r} - v_{1l}}\right) \tag{1.21}$$

Since the radius r is the same for all wheels, the angular velocities can be substituted for the tangential velocities, obtaining:

$$R_1 = \frac{i}{2} \cdot \frac{\omega_{1r} + \omega_{1l}}{\omega_{1r} - \omega_{1l}} \tag{1.22}$$

This relationship shows how the radius of curvature, and consequently the trajectory generated, depends on the ratio of the angular velocities of the wheels, while the value of the individual ve-

locities affects the travel times of the trajectory. Knowing the radius of curvature, the curvature referred to the front end can be expressed as:

$$\rho_1 = \frac{1}{R_1} = \frac{2}{i} \cdot \frac{\omega_{1r} - \omega_{1l}}{\omega_{1r} + \omega_{1l}} \tag{1.23}$$

If we consider the point O_1, the distance between CIR_1 and O_1 corresponds to the radius of curvature R_1, so we can state that:

$$v_1 = R_1 \dot{\varphi}_1 = \frac{\dot{\varphi}_1}{\rho_1} \tag{1.24}$$

This relationship is extremely important because it allows linking the longitudinal velocity to the angular velocity of the front module through the radius of curvature of the latter. The second consideration concerns the sign of the curvature ρ_1; the convention adopted provides:

- **Positive curvature** ($|\omega_{1r}| > |\omega_{1l}|$) if the robot rotates counter-clockwise in forward gear.
- **Negative curvature** ($|\omega_{1r}| < |\omega_{1l}|$) if the robot rotates clockwise in forward gear.
- **Zero curvature** ($|\omega_{1r}| = |\omega_{1l}|$) if the robot follows a straight trajectory.

1.1.3 System Equilibrium Angle and the Problem of Jackknifing

By analyzing equation (1.12), it can be noted that the robot has a certain equilibrium angle δ, that is, a value of δ such that $\dot{\delta} = 0$.

This analysis is the primary defense against the jackknife phenomenon, a catastrophic instability where the articulation angle δ increases without bound. This causes the trailer to swing around and fold acutely onto the tractor, representing a complete loss of control that the system must be designed to avoid.

The first step to avoid the jackknife phenomenon is to determine this equilibrium angle, which varies depending on the trajectory. Two possible cases exist:

- Rectilinear trajectory ($\dot{\varphi}_1 = 0$).
- Circular trajectory ($\dot{\varphi}_1 \neq 0$).

Rectilinear Trajectory

To determine the equilibrium angle δ_{eq}, equation (1.12) is set to 0.

$$\dot{\delta} = \left(\frac{a}{b}\cos\delta_{eq} + 1\right)\dot{\varphi}_1 - \frac{1}{b}\sin\delta_{eq} \cdot v_1 = 0 \tag{1.25}$$

If the robot goes straight ($\dot{\varphi}_1 = 0$) then the equation simplifies to:

$$-\frac{1}{b}\sin\delta_{eq} \cdot v_1 = 0 \implies \sin\delta_{eq} = 0 \tag{1.26}$$

Thus, two possible solutions are obtained:

$$\delta_{eq,1} = 0 \quad \text{or} \quad \delta_{eq,2} = \pi \tag{1.27}$$

The solution $\delta_{eq,1} = 0$ corresponds to having the front and rear modules aligned, while the solution $\delta_{eq,2} = \pi$ corresponds to having the front and rear modules overlapping. It can be verified that, if the mobile robot is moving in reverse along a pre-established trajectory, the angle δ diverges, and if the reverse path exceeds a certain length, the two modules will collide, leading to the jackknife phenomenon. Stating that in reverse the angle δ increases is equivalent to saying that the solution $\delta_{eq,1} = 0$ represents an unstable equilibrium condition for the robot, which instead tends to move towards its stable equilibrium condition in reverse, that is, to the solution $\delta_{eq,2} = \pi$.

Having arrived at this point, and having detailed the complexity of the Kinematic Model of TTS, it is correct to remember that the objective of this study is not to develop an anti-jackknifing controller. While system stability is a critical topic, it falls out of the scope of this thesis. The work's contribution is instead an obstacle avoidance module for forward movement. The inherent stability of forward motion, which ensures the system tends towards $\delta = 0$, allows us to focus on the path planning challenge without addressing the control problems associated with reverse maneuvers.

1.2 The Autonomous Navigation Pipeline: Perception, Planning, and Control

Modern robotics depends on machines that can operate autonomously between a starting point and a destination through unassisted navigation. The complex process of autonomous navigation requires breaking down into separate modules, which form the autonomous navigation pipeline or stack. The modular design of this system enables a methodical solution to complex problems because each module performs a specific function. The Perception-Planning-Control loop represents the standard pipeline structure, which enables robots to "sense the world, and its own state, create a model of the world, and plan and execute a path to a goal location" [2].

1. Perception

The perception module functions as the robot's sensor system which enables it to interact with its environment. The main responsibility of this module consists of gathering raw data from a suite of onboard sensors and processing it into a meaningful and structured representation of the environment. For an agricultural rover, this involves detecting both environmental features and agricultural field elements, including crop rows and headlands, and un-farmable sections. The most common sensors for these operations include:

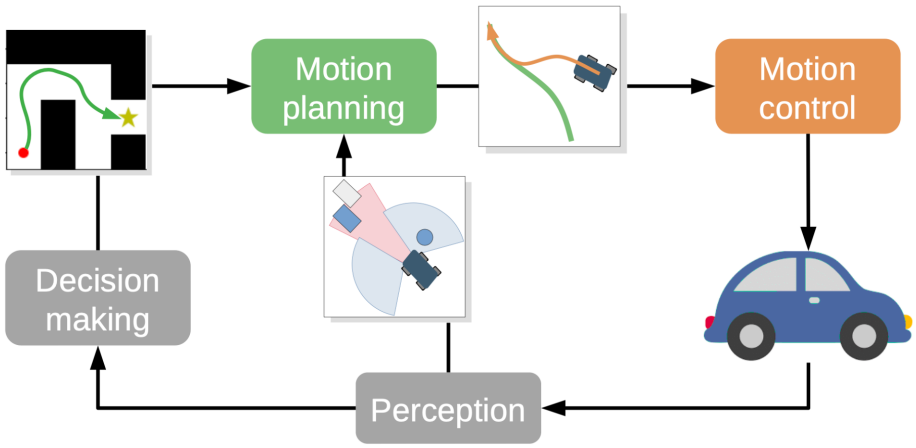


Figure 1.3: The Perception-Planning-Control pipeline with arrows showing the flow of information between modules.

- **LiDAR (Light Detection and Ranging):** The Light Detection and Ranging technology, known as LiDAR sensors, use laser beams and their reflection to measure distances and creates a three-dimensional point cloud of environmental data. This is crucial for accurate obstacle detection and mapping.
- **Cameras (Monocular, Stereo, and RGB-D):** Offer rich visual information. "Stereo cameras can provide depth information through triangulation, while RGB-D cameras add a depth channel directly. This data is fundamental for semantic segmentation (e.g., distinguishing crops from weeds) and visual odometry". [3]
- **GPS (Global Positioning System) and IMU (Inertial Measurement Unit):** These sensors are vital for localization. GPS provides a global position estimate and data, which RTK (Real-Time Kinematic) technology enhances to deliver centimeter-grade precision. The IMU tracks the robot's orientation and angular velocity to calculate its position between GPS updates while reducing sensor drift.

The output of the perception module is typically a world representation, such as an occupancy grid map. This is a popular technique that generates a discretized representation of the environment where each cell in a grid is assigned a probability of being occupied, free, or unknown. This map forms the basis upon which the planning module will operate.

2. Planning (or Decision Making)

The planning module acts as the "brain" of the Robotic system. It receives a representation of the environment from the perception module and is responsible for determining a safe and efficient path to the goal. This process is often hierarchical, broken down into several layers:

- **Mission Planning:** At the highest level, a mission planner might decide on a sequence of high-level tasks, such as "cover the entire field" or "navigate to the refueling station."

- **Global Path Planning:** This layer generates a complete path from the robot's current location to the goal, typically assuming an environment and complete knowledge of the space as demonstrated in our case study. The system's objective is often to find an optimal route based on given criteria like distance, energy consumption, or waypoints compliance.

- **Local Path Planning (or Motion Planning):** This layer is responsible for handling real-time adjustments, obstacle avoidance, or path modification. It generates a brief, kinematically feasible trajectory that ensures the robot's movements are smooth, safe, and respect its physical constraints while reacting to dynamic obstacles or newly perceived information.

3. Control

The final module in the pipeline is the controller. This performs the task of carrying out the trajectory that the planner produces. The controller receives the desired trajectory (a sequence of poses or velocities) and calculates the necessary actuator commands—such as the specific angular velocities for the left and right tracks of a differential drive robot—to make the robot follow the path as closely as possible. "A low-level motion controller acts as the bridge between the abstract plan and the physical hardware, translating desired motions into concrete actions" [2] The controller must continuously compensate for discrepancies between the robot's actual state and the desired state. These errors can arise from many different phenomena, such as wheel slippage, uneven terrain, sensing errors, or modeling inaccuracies. This continuous correction is achieved through a feedback loop, where the robot's current state (provided by the first two levels of the autonomous driving pipeline) is constantly compared against the reference trajectory. Following this pipeline, the autonomous tractor is therefore able to achieve robust and adaptive navigation in complex changing environments through its continuous perception-planning-control cycle, making it suitable for agricultural operations.

1.3 Path Planning in Robotics: From Graph-Search to Sampling-Based Methods

Mobile robotics depends on path planning to determine a continuous set of robot configurations that lead from the starting point to the target location while avoiding collisions with obstacles. The problem complexity is dictated by the dimensionality of the robot's Configuration Space (C-Space). The C-Space is an abstract space representing all possible configurations (positions and orientations) of the robot. In this space, the robot is represented as a single point, and obstacles from the real world are "inflated" to represent all the robot configurations that would result in a collision. Planning a collision-free path for the robot in the real world is thus equivalent to planning a path for a point in the free C-Space.

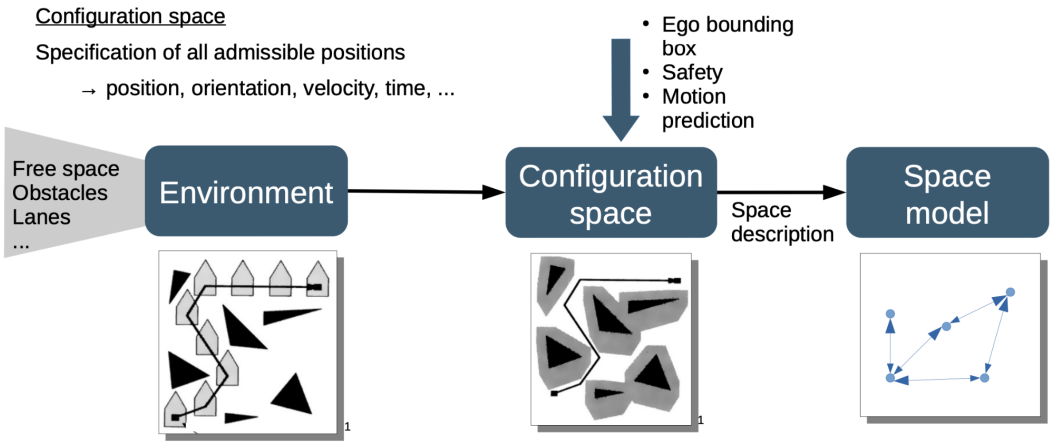


Figure 1.4: The transformation of a 2D world with a robot and obstacles into a Configuration Space where the robot is a point and obstacles are inflated.

Path planning algorithms have traditionally been divided into distinct categories throughout history. The fundamental path planning methods include graph-search planners, which represent the earliest approaches. With graph-search planners, free C-space gets divided (discretized) into a graph structure, where nodes represent locations and edges represent valid connections between them. The first two algorithms to be widely adopted are Dijkstra's and A*, and their efficacy makes them massively utilized to search for the shortest path in graph configurations. Ultimately, even if such algorithms guarantee optimality (finding the shortest path if one exists), these methods become computationally intractable in high-dimensional C-spaces due to the exponential growth of the number of nodes required, a phenomenon cited in literature as the "curse of dimensionality". [4] To overcome this, sampling-based planners were developed. "Instead of trying to deterministically explore the whole configuration space, sampling-based algorithms try to capture the connectivity of the space by sampling it with a number of random points" [5]. They are probabilistically complete, meaning that if a path exists, the probability of finding it approaches 1 as the number of samples increases.

1.3.1 Probabilistic Roadmaps (PRM)

The Probabilistic Roadmap (PRM) is a classic and powerful sampling-based algorithm that operates in two distinct phases: a learning phase (or construction phase) and a query phase.

1. Learning Phase (Roadmap Construction)

This phase builds a graph, or roadmap, that captures the connectivity of the free C-space. This is an offline process performed once for a given environment. The core philosophy behind this modus operandi, as stated by its creators, is that "while it is difficult to find a path in a high-dimensional configuration space, it is relatively easy to check whether a given configuration is

in free space and whether a simple path, for example a straight line segment, is collision-free" [6]. This philosophy not only makes it easy to implement, but it makes PRM perfect for open space navigation, as a creation of a complete graph in space can be avoided. The steps are:

- **Random Sampling:** A large number of configurations (nodes) are randomly sampled from the C-space.
- **Collision Checking:** Each sampled configuration is tested for collision. If the robot at that configuration would collide with an obstacle, the sample is discarded. Otherwise, it is added as a node to the roadmap.
- **Node Connection:** For each valid node, the algorithm attempts to connect it to its k-nearest neighbors using a simple local planner (e.g., a straight line). If the connection is collision-free, an edge is added to the roadmap. The weight of the edge is usually the Euclidean distance between the configurations.

2. Query Phase

Once the roadmap is built, it can be used to solve multiple path planning problems within that environment. For a given start and goal configuration:

- **Connect to Roadmap:** The algorithm attempts to connect the start and goal configurations to their nearest neighbors in the pre-computed roadmap, just as in the construction phase.
- **Graph Search:** With the start and goal now part of the roadmap, a graph-search algorithm like Dijkstra's is used to find the shortest sequence of edges connecting the start node to the goal node.

The primary advantage of PRM is its efficiency in high-dimensional spaces and its ability to solve many queries after the initial roadmap construction. However, it wouldn't be optimal for dynamic environments where obstacles move if no obstacle avoidance protocol is present, as the entire roadmap would need to be recomputed.

1.4 Trajectory Tracking: A Review of Core Strategies

Once a geometric path has been generated by the planner, the task of the trajectory tracking controller is to compute the wheel velocities required to follow this path accurately. This is a crucial step of the navigation pipeline, as a generated path could be theoretically optimal, but even so meaningless if the robot cannot physically execute it. The performance of these controllers is often measured by their ability to minimize the lateral error, which is the perpendicular distance between a point on the robot (e.g., the rear axle) and the nearest point on

the reference path. A variety of control strategies have been developed and applied in mobile robotics, ranging from classical control systems, passing simple geometric methods to more complex model-based approaches specifically designed for navigation.

Classic Feedback Control: PID Controllers

A classical and arguably most common approach to all control problems is the Proportional-Integral-Derivative (PID) controller. When applied to path tracking, a PID controller calculates an output (e.g., a steering command) based on the lateral error. It works by summing three components:

- **Proportional (P):** Proportional to the current lateral error. A larger error results in a stronger corrective action.

- **Integral (I):** Accumulates past errors over time. This helps eliminate steady-state error, ensuring the robot eventually stabilizes perfectly onto the path.

- **Derivative (D):** Proportional to the rate of change of the error. This term dampens and "smooths" the response, reducing overshoot and oscillations.

While simple and often effective for most control problems, tuning the three PID gains (Kp, Ki, Kd) can be challenging. Moreover, as the lateral error calculation is dependent solely on position sensing, its performance can degrade at high speeds or in paths with high curvature. This usually shifts the choice of a Tractor-trailer system's controller to other techniques that either exploit the geometry or the state-space configuration of the robotic platform.

Geometric Controllers: Pure Pursuit and Stanley

Geometric controllers offer an intuitive and computationally efficient alternative, relying on the kinematic relationship between the robot and the path.

- **The Pure Pursuit Controller:** This popular algorithm calculates the curvature of a circular arc that connects the robot's current position to a target point on the path, a certain lookahead distance ahead. The choice of this distance is a critical trade-off: "a large lookahead distance will produce a smooth path but will cut corners, whereas a small lookahead distance will track the path more accurately but can cause oscillations" [7].

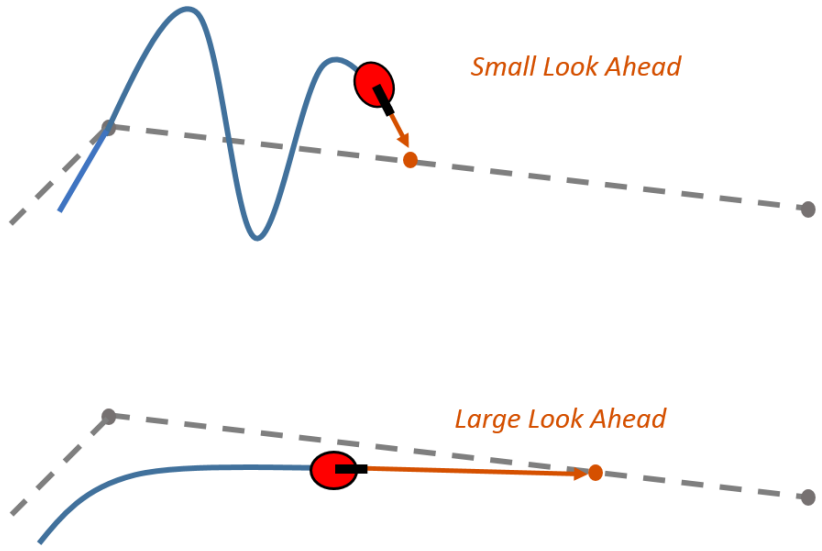


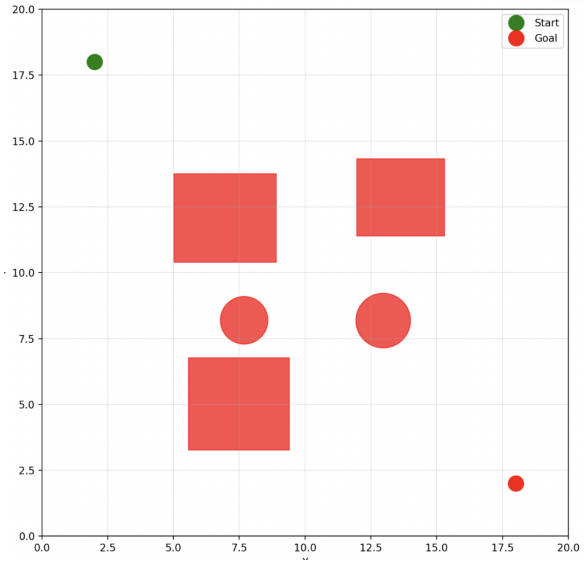
Figure 1.6: Illustration of the Pure Pursuit controller with a robot, a path, and the lookahead distance to a target point. [8]

- **The Stanley Controller:** This controller was developed by Stanford University for the DARPA Grand Challenge, but its versatility and simplicity render it another widely appreciated and implemented geometric method. Unlike Pure Pursuit, which only considers a future goal point, the Stanley controller computes a steering command based on two criteria: the lateral error at the front axle and the heading error (the difference between the vehicle's orientation and the path's orientation). This often leads to more aggressive and precise tracking, especially in correcting errors quickly.

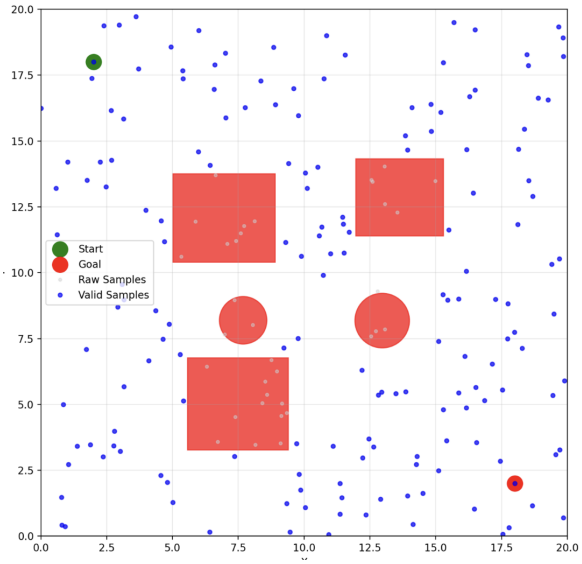
Model-Based Controllers: Model Predictive Control (MPC)

For systems with complex but known dynamics, Model Predictive Control (MPC) is a powerful online optimization-based strategy. MPC uses a dynamic model of the robot to predict its future trajectory over a short time horizon and optimizes the control inputs to minimize path deviation while respecting system constraints [9]. Its ability to "predict the future evolution of the system makes it possible to solve the control problem online, at each sampling instant" [10], making it highly reactive and suitable for handling the unstable dynamics of articulated vehicles, and apt for dynamical environments such as Urban settings or highways. This controller is nowadays the standard for research and application in autonomous driving solutions.

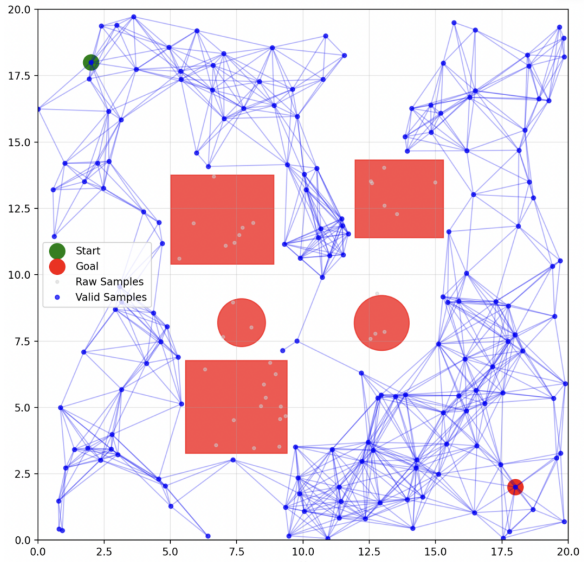
While several controllers have been introduced here, a more detailed mathematical formulation and comparative analysis, particularly focusing on the implementation of Pure Pursuit and PID controllers for the agricultural rover, will be the subject of the following chapters.



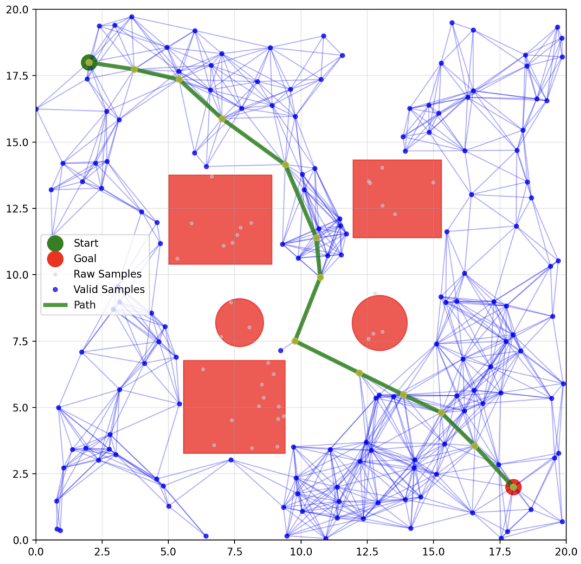
(a) Obstacles in a space with start and goal.



(b) Randomly sampled nodes and samples.



(c) Constructed roadmap showing connections.



(d) Final path found by Dijkstra's algorithm.

Figure 1.5: The PRM construction process showing: (a) obstacles in a space with start and goal, (b) randomly sampled nodes and valid samples, (c) the constructed roadmap with connected, collision-free nodes, and (d) the final path found by Dijkstra's algorithm.

Chapter 2

System Modeling and Path Generation Framework

This chapter lays the groundwork for the core research of this thesis. It discusses the computational environment, the mathematical models used to represent the agricultural rover and its operating environment, and the required steps for generating an initial, kinematically naive reference path. The output of this framework is a smoothed, geometrically valid trajectory which serves as the input for the hazard detection and modification systems developed in Chapter 3. This initial path represents a crucial step: it is a solution that a standard planner would deem "correct" yet it retains significant latent risks for an articulated vehicle. Understanding its generation process is essential to appreciate the necessity of the subsequent safety-aware modification layers.

2.1 Development Environment: MATLAB and Robotics Toolbox

For this project, MATLAB R2024a was selected as the primary platform. The choice was driven by the core features of the MATLAB environment, both as an established standard in academic and industrial engineering fields, and for its suitability for tasks involving numerical computation, algorithm development, and data visualization[11]. Its core strength lies in its matrix-based language, which allows for the expression of complex mathematical operations in a syntax that is both concise and intuitive [11].

To further accelerate the development process, this work leverages two key MATLAB add-ons: the **Robotics System Toolbox™** and the **Navigation Toolbox™**, which are validated, industry-standard tools. These toolboxes provide a rich library of pre-built, optimized functions and objects for common robotics and autonomous navigation tasks. Their integration was essential in facilitating the development of the path correction framework, allowing the research to concentrate on the novel aspects of the hazard detection and path modification algorithms. It's fair to mention that for the scope of this research, lower-level implementations of baseline algorithms were needed rather than the ones included in the two toolboxes, rendering the research even more "didactical" and instructing. The key components from these toolboxes that

were instrumental to this project are:

- `binaryOccupancyMap`**:** As introduced in Chapter 1, a robust representation of the environment is a prerequisite for any path planning task. The `binaryOccupancyMap` object provides a highly efficient and structured method for this. It discretizes the continuous two-dimensional world into a grid, where each cell stores a binary value representing its state: 0 for free space and 1 for occupied space. This data structure is optimized, making collision checks for points or sets of points exceptionally fast. Additionally, an exceptional feature of "map" objects is their capability to be "inflated," by adding a desired thickness to the walls of the environment. This is instrumental for path generation, as we can massively simplify the path planner by feeding it an inflated map having a desired safety bound, instead of hard-coding the planner with a "smarter" sampling phase. This ensures that all the nodes are fetched with a minimal desired distance while keeping the random nature of the planner untouched.

- `vehicleCostmap`**:** While the `binaryOccupancyMap` is sufficient for determining absolute collisions, a more nuanced representation of risk is often required. The `vehicleCostmap` object was used in the path modification stage as a background check for the correctness of the protocol (detailed in Chapter 3), without exploiting its tremendous capabilities. It generates a cost grid where the value of each cell represents the cost (or risk) for a vehicle's specific footprint to be centered at that location. It inherently accounts for the vehicle's geometry, creating a "cost field" that is low in open space and high near obstacles. This allows for the evaluation of "close calls" in addition to hard collisions, a crucial element for ensuring the final modified path is not only collision-free but also maintains a safe operational margin.

- `inflationCollisionChecker`**:** To interface the custom vehicle geometry with the costmap and collision-checking utilities, the `inflationCollisionChecker` object was again used as a background check of the validity of the heart of the research. It allows the definition of a complex vehicle shape (like a rectangle) and then abstracts it into a set of covering circles. This abstraction is a common technique in robotics, as circle-based collision checking is computationally far less expensive than polygon-based methods [4]. This tool provided an efficient bridge between our custom vehicle model and MATLAB's built-in safety validation functions.

By employing these established tools and interfacing them with more standard geometric solutions, the project could ensure that, especially in early development stages, the foundational layers of environmental representation and collision checking were robust and efficient, thereby dedicating the bulk of the research effort to the primary contribution: the development of novel hazard detection methodologies for articulated vehicles.

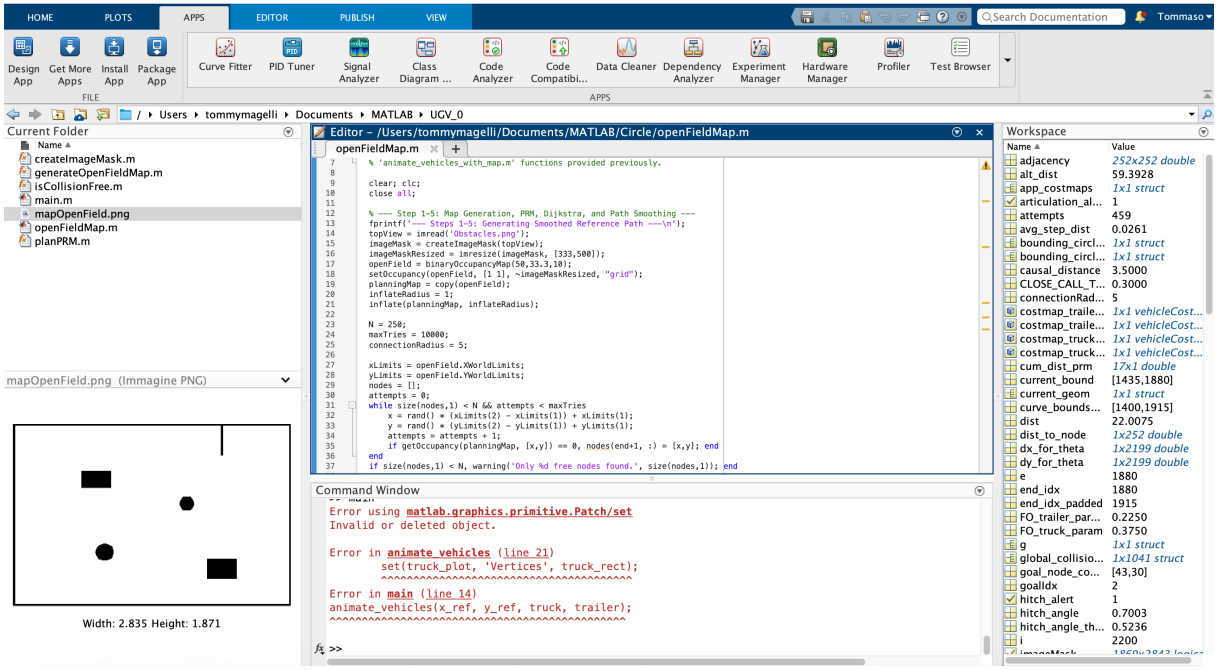


Figure 2.1: The MATLAB development environment. The platform's integrated visualization and specialized toolboxes for robotics and navigation provided a powerful framework for algorithm prototyping and validation.

2.2 Modeling the Agricultural Rover and the Open-Field Environment

A meaningful simulation requires accurate digital models (twins) of both the robot and its world. The fidelity and coherence of these models directly impacts the validity of the research findings. This section describes the specific implementation details of how the open-field environment and the articulated agricultural rover were represented digitally.

Environment Modeling

The operational environment is modeled as a two-dimensional planar world, a common and effective simplification for many ground vehicle navigation tasks. For the sake of simplicity, a simple black and white environment with spikes, obstacles, and narrow passages was generated. The process, implemented in the initial steps of the `openFieldMap.m` script, transforms a visual representation of a field into a structured data format suitable for algorithmic processing.

1. Image-to-Map Conversion

The workflow originates with a top-down image of the field (e.g., `Obstacles.png`). This image is loaded into MATLAB using `imread`, and a simple color-based thresholding is applied in the `createImageMask` helper function to generate a binary mask. This mask effectively segments

the image into two classes: traversable terrain and non-traversable obstacles, respectively white and black.

2. Occupancy Grid Representation

This binary mask is then ingested by the `binaryOccupancyMap` constructor. In this project, the map was initialized with specific world dimensions and resolution, as shown in the code:

```
% Width(m), Height(m), Resolution(cells/m)
openField = binaryOccupancyMap(50, 33.3, 10);
setOccupancy(openField, [1 1], ~imageMaskResized, "grid");
```

This creates a grid where each cell represents a $0.1m \times 0.1m$ square of the real world. The choice of resolution is a critical trade-off: "a higher resolution provides a more detailed representation of obstacle boundaries but increases the memory footprint and computational cost of all subsequent operations" [12]. The chosen resolution of 10 cells/meter was deemed a suitable balance for this application.

3. Map Inflation for Safety Buffering

A practical and standard practice in path planning is to work with a "configuration space" representation where obstacles are inflated to account for the robot's physical size. A simplified inflation is applied to the base map to create an initial safety buffer.

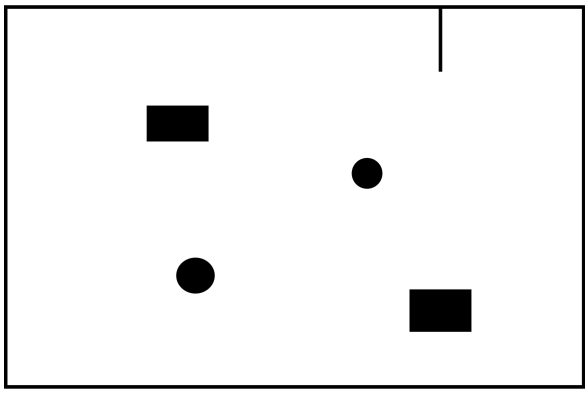
```
planningMap = copy(openField);
inflateRadius = 1; % meters
inflate(planningMap, inflateRadius);
```

This operation expands the boundaries of all obstacles by a 1-meter radius. The initial PRM planner operates on this `planningMap`, which ensures that the generated path inherently maintains a minimum distance from obstacles, making it safer and providing more "room to maneuver" for the subsequent modification stages.

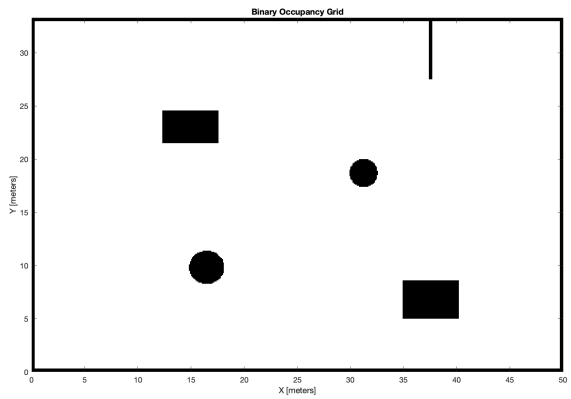
Vehicle Modeling

The articulated rover is represented by a kinematic model. This model describes the geometry of motion but deliberately omits dynamic effects like inertia, friction, and slip. This simplification is justified because the primary challenge being addressed—off-tracking—is fundamentally a kinematic phenomenon. The vehicle's physical properties are centralized in the `geom()` support function, which returns a struct g containing the essential parameters:

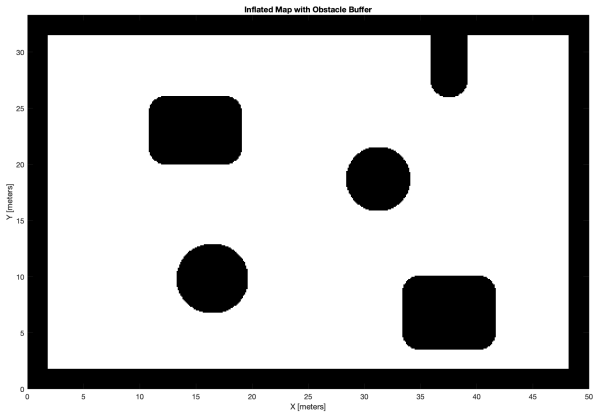
- `g.Ltr = 2.50; g.Wtr = 1.40;` % Tractor Length and Width (m)
- `g.Ltl = 1.50; g.Wtl = 1.45;` % Trailer Length and Width (m)



(a) Raw image of the environment.



(b) Binary occupancy map.



(c) Inflated planning map.

Figure 2.2: Environment modeling process. From a raw image (a), a binary occupancy map is created (b), which is then inflated (c) to create a conservative planning space with safety buffers around obstacles.

- `g.Lh = 2.00;` % Hitch Length (m), from tractor rear axle to trailer pivot
- `g.v = 0.50; g.dt = 0.05;` % Constant velocity (m/s) and time step (s)
- `g.look = 1.00;` % Lookahead distance for Pure Pursuit (m)

The `simulate_vehicle.m` function uses these values to propagate the state of the vehicle. The tractor's motion is determined by a Pure Pursuit controller, and the trailer's motion is passively derived from the tractor's position and the geometric constraint imposed by the hitch.

2.3 The Path Generation Workflow: From PRM to a Smoothed Path

Once the environment and vehicle models are established, the framework generates a globally valid, smooth path. The `openFieldMap.m` workflow converts a high-level navigation problem

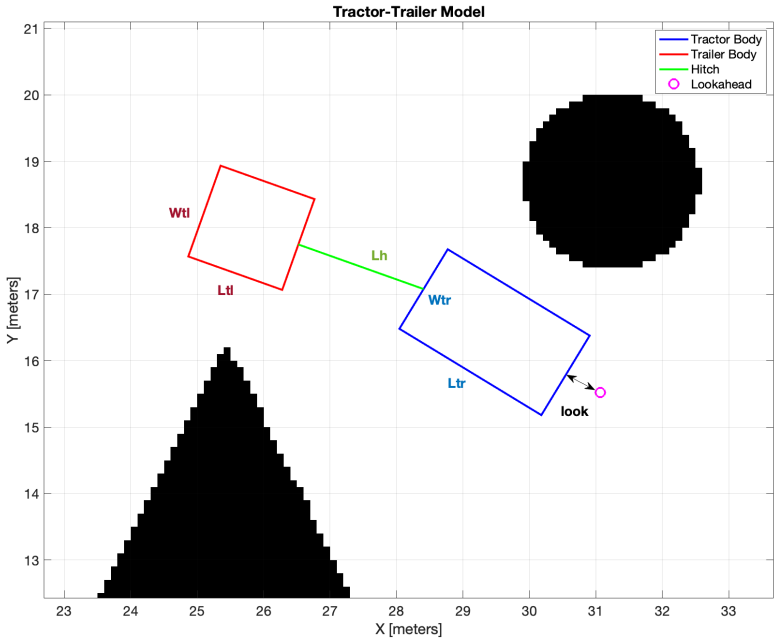


Figure 2.3: Kinematic model of the articulated rover, showing the key geometric parameters (`Ltr`, `Ltl`, `Wtr`, `Wtl`, `Look` and `Lh`) that govern its motion and articulation.

into a concrete, continuous reference trajectory. This process consists of three consecutive operations: initial path planning, densification, and smoothing.

1. Initial Path Planning with Probabilistic Roadmap (PRM)

The implementation follows the classic two-phase approach:

- **Construction Phase:** The algorithm builds a roadmap by randomly sampling N nodes within the world limits. Each valid node is checked against the inflated `planningMap`. The algorithm then attempts to connect pairs of valid nodes with a straight-line edge if they are within a specified `connectionRadius` and the path between them is collision-free.

- **Query Phase:** The start and goal coordinates are added to the graph, and connections are established with nearby nodes. Then, Dijkstra's algorithm is used to determine the shortest path.

The output is `path_indices`, a sequence of straight-line segments from the start to the goal.

2. Path Refinement: Densification and Smoothing

The path produced by PRM is a piecewise-linear series of waypoints. To transform this into a kinematically "easy" trajectory, a two-step refinement process is applied.

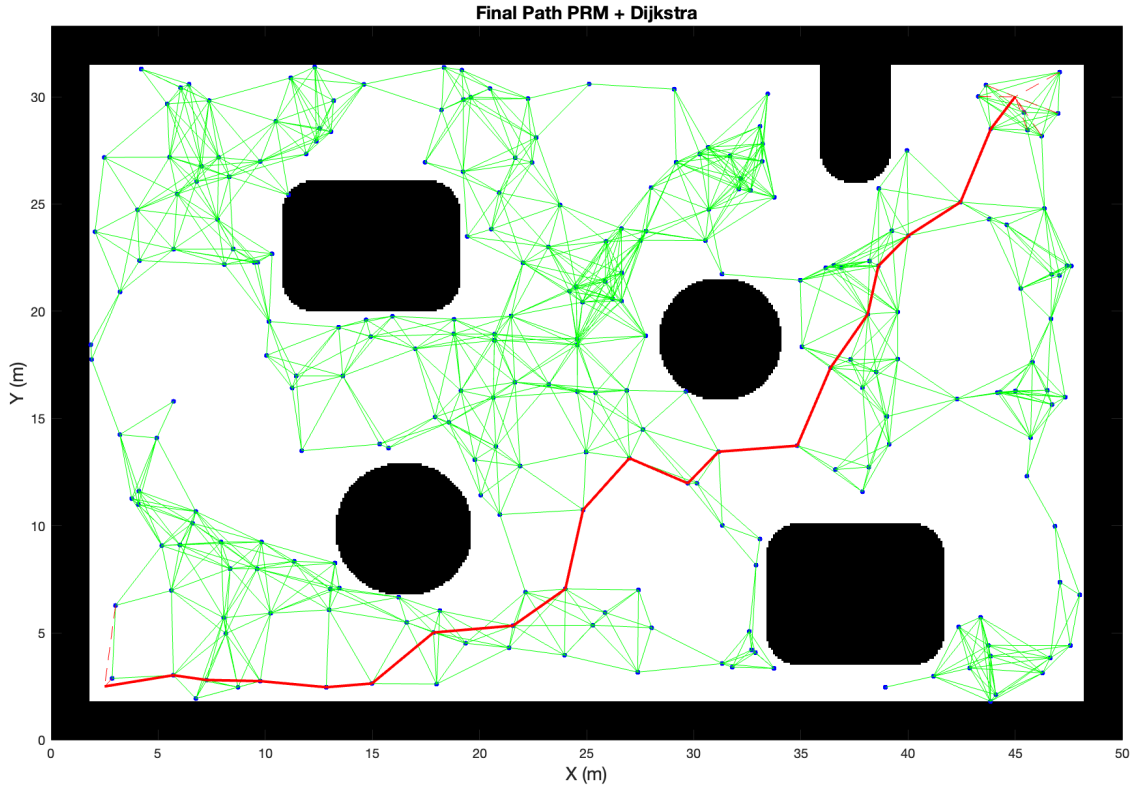


Figure 2.4: The PRM path generation process. Randomly sampled nodes (blue) are connected to form a roadmap (green), which is then searched by Dijkstra's algorithm to find an optimal coarse path (red).

- **Densification:** The first step increases the number of points using a Piecewise Cubic Hermite Interpolating Polynomial (`pchip`) via MATLAB's `interp1` function [11].

```
num_interp_points = 2200;
interp_dist_points = linspace(0, cum_dist_prm(end), num_interp_points);
x_ref = interp1(cum_dist_prm, x_ref_prm, interp_dist_points, 'pchip');
```

  The `'pchip'` method is chosen because it is shape-preserving and will not "overshoot" the data points, which prevents the interpolated path from introducing new curves that might cut too close to obstacles (Fritsch & Carlson, 1980).

- **Smoothing:** The final step ensures the path is sufficiently smooth (ideally C^2 continuous) using a multi-pass Gaussian smoothing filter via the `smoothdata` function.

```
x_ref = smoothdata(smoothdata(x_ref, 'gaussian', 60), ...
                   'gaussian', 45), 'gaussian', 25);
y_ref = smoothdata(smoothdata(y_ref, 'gaussian', 60), ...
                   'gaussian', 45), 'gaussian', 25);
```

  This iterative method first uses a large window (60) to eliminate major jaggedness, then

smaller windows (45 and 25) to remove high-frequency noise without drastically altering the path's global shape.

The final product is a set of vectors, `x_ref` and `y_ref`, representing a dense, smooth, and geometrically valid reference trajectory. However, this path is "kinematically naive" as it was generated without consideration for the articulated nature of the rover.

2.4 Overview of the Offline Path Modification Framework

The smoothed reference path is fundamentally insufficient and potentially dangerous for an articulated vehicle due to the off-tracking effect. To address this, this thesis proposes an offline path modification framework. The entire workflow of analysis and correction is performed before the vehicle begins any physical movement. The framework is implemented as a logical cycle and consists of four distinct phases:

1. **Phase 1: Initial Simulation and Data Generation:** The framework starts by running a precise simulation of the kinematic vehicle model across the reference path using the `simulate_vehicle` function. This produces a detailed time-indexed record of the exact positions and orientations of both the tractor and trailer.

2. **Phase 2: Hazard Detection (The Core Contribution):** This module receives the simulation data and evaluates which parts of the path lead to dangerous situations. Two distinct, deterministic methods were developed for this purpose: one based on reacting to simulated collisions, and another based on proactively identifying high-risk maneuvers. The specific algorithms are the core topic of Chapter 3. The output is a set of start and end indices that frame the identified hazardous segments.

3. **Phase 3: Trajectory Modification with Bézier Curves:** The identified hazardous segments are replaced. The `modify_curves_bezier` function generates a new, safe alternative for each dangerous segment using a C^2 continuous Bézier curve. The shape of this curve is optimized to connect the safe entry and exit points while robustly avoiding obstacles.

4. **Phase 4: Final Validation and Verification:** After all hazardous segments have been replaced, a new modified reference path is created. To close the loop, this final path is subjected to another full simulation to confirm that all previously identified hazards have been eliminated and no new dangerous behaviors have been introduced.

This four-stage offline framework delivers a complete solution that transforms any "kinematically naive" path into a safe route for an articulated vehicle in unstructured environments.

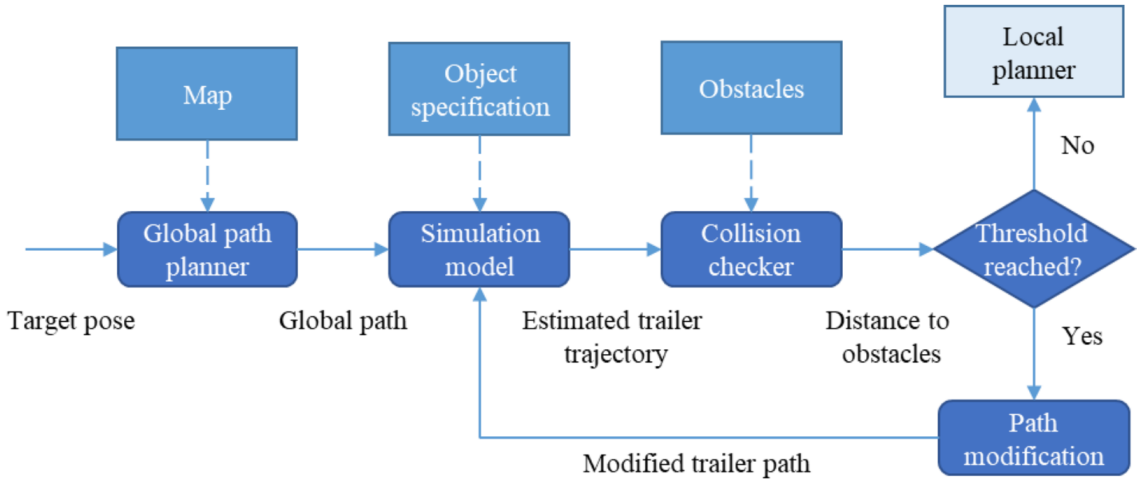


Figure 2.5: The offline path modification framework. This iterative process simulates a naive path, detects hazardous segments, replaces them with safe alternatives, and validates the final result before execution.

Chapter 3

Development of the Trajectory Modification Framework

Having generated a kinematically naive reference path, as detailed in Chapter 2, the central challenge of this thesis can now be addressed: how to transform this path into one that is certifiably safe for an articulated vehicle. This chapter presents the design and implementation of the complete offline modification framework, showcasing different methodologies implemented to reach the desired result. The proposed solution is ideated following the core philosophy of proactive analysis: it identifies and resolves potential dangers before the vehicle begins motion.

The first portion of the third chapter is dedicated to the introduction of Bézier curves, which are the mathematical tool chosen for path modification. As their properties make them the most suitable tool for this path, it's important to stress their importance. This chapter then dives into the primary contribution of this work: a detailed exposition of the two novel, deterministic hazard detection methodologies. Following this, it describes the critical "Maneuver Window" algorithm, which correctly associates a hazardous effect with its root cause. Finally, it details the informed replanning process, where new path segments are not just generated but are rigorously validated with a sophisticated risk model to guarantee their safety.

3.1 Bézier Curves: The Mathematical Tool for Trajectory Replanning

The task of modifying a path segment requires a tool that can generate a new trajectory that is not only safe but also smooth and seamlessly integrated with the existing path, meaning that it does not create continuity problems when stitched to the path. For this purpose, cubic Bézier curves were selected as the most suited geometric primitive that could serve the research scope. A cubic Bézier curve is a parametric curve defined by a set of four control points: a start point (P_0), an end point (P_3), and two intermediate control points (P_1 and P_2) that act as "handles" that have a direct effect on the curve's shape.

The choice of Bézier curves is not arbitrary; as stated by Foley, they possess a unique combination of mathematical properties that make them exceptionally well-suited for vehicle trajectory generation, a fact well-documented in robotics and computer graphics literature [13].

1. **Endpoint Interpolation:** A key property is that the curve is guaranteed to pass through its start and end control points (P_0 and P_3). This is essential for the task of "stitching" a modified segment back into the original path. By setting P_0 and P_3 to the safe entry and exit points of a hazardous segment, a seamless, continuous, differentiable connection is ensured.

2. **C² Continuity:** Bézier curves are infinitely differentiable (C∞ continuous) within their own domain (between P_0 and P_3). When properly constructed, they can be used to generate paths that are C² continuous, meaning they have continuous curvature. As established in Chapter 2, this is a critical requirement for non-holonomic vehicles, as these kinds of paths avoid instantaneous changes in steering angle, leading to stable and predictable motion.

3. **The Convex Hull Property:** This is arguably the most powerful property from a safety perspective. The entire Bézier curve is always contained within the convex hull of its four control points. As noted by Park, "This property is useful for fast collision checking. If the convex hull of the control points is in a collision-free space, the path is guaranteed to be collision-free" [14]. This provides an extremely powerful heuristic: by placing the control points in safe regions, we can have a high degree of confidence that the resulting path will also be safe.

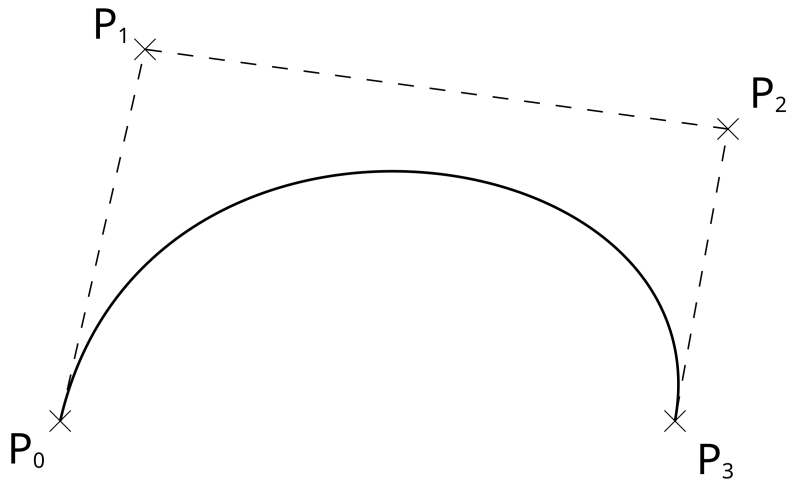


Figure 3.1: A cubic Bézier curve defined by four control points. The curve interpolates the start and end points and is fully contained within the convex hull (dotted line), providing a powerful guarantee for collision avoidance. From [15].

Implementation: The Recker-Bézier Method

To apply Bézier curves to vehicle paths, a structured method for placing the control points is needed. This project adopts a methodology similar to that proposed by Recker et al. (2021)

for modeling curve modification segments. This article and its novelties have served as the main inspiration and guideline for the development of the path modification heuristic of this thesis, which, together with the hazard detection portion, sets the base for the obstacle avoidance framework. [16]

The control points for a segment are defined as follows:

- P_0 **and** P_3**:** The start and end points of the nominal segment to be replaced. Their positions are fixed.
- P_1 **and** P_2**:** These points are placed along the tangent vectors at the start and end points, respectively. The tangent vector at P_0 is defined by the direction of the path leading into the segment, and the tangent at P_3 is defined by the path's direction exiting the segment. The control points P_1 and P_2 are then placed at distances d_1 and d_2 along these tangents.

The shape of the curve is therefore controlled by just two scalar parameters, d_1 and d_2. The core of the path modification algorithm, implemented in the `apply_recker_curve` nested function within `modify_curves_bezier.m`, is an iterative search. It systematically tries different combinations of d_1 and d_2 to find a Bézier curve that minimizes a cost function representing collision risk, ultimately finding the safest possible alternative trajectory.

3.2 Deterministic Hazard Detection Methodologies

The most critical component of the framework is its ability to analyze the initial simulation and pinpoint which parts of the reference path are dangerous. This is not a trivial task; the danger may not be an obvious collision but a subtle maneuver that precedes it. To address this, two distinct, deterministic detection methodologies were developed and implemented. They represent two opposing but complementary philosophies: a reactive, geometrically exact approach and a proactive, conservative approach.

Method A: Collision-Based Detection via Precise Swept-Path Envelope

This first method defines a path segment as hazardous if, and only if, a high-fidelity simulation shows a physical collision between the trailer's body and an obstacle. It does not guess or predict risk; it measures the actual outcome of the naive path. This approach is implemented in the script version that focuses on envelope-based detection. The process unfolds in two parallel steps during the initial simulation:

1. **Precise Envelope Construction:** The algorithm constructs the true "swept-path envelope" of the trailer. At each time step `i` of the simulation, it calculates the four corner coordinates of the trailer's rectangular footprint in the global frame (`world_corners`). This new patch of space is then merged into a master envelope using MATLAB's `union` function.

```
% Inside the simulation loop
current_patch = polyshape(world_corners([3,1,2,4],:));
swept_path_envelope = union(swept_path_envelope, current_patch);
```

The final `swept_path_envelope` object is a single, complex polygon that represents every point in the environment that was occupied by the trailer at any point during the maneuver.

2. **Point-Based Collision Detection:** While the envelope is being built, a direct collision check is performed. For each corner of the trailer at each time step, the `getOccupancy` function is called to check its status against the `openField` map.

```
% Inside the simulation loop
for j = 1:4
    if getOccupancy(openField, world_corners(j,:)) == 1
        is_colliding(i) = true;
        break; % A single colliding corner is sufficient
    end
end
```

This populates a boolean vector, `is_colliding`, where `is_colliding(i)` is true if any part of the trailer was in collision at the i-th step.

The primary advantage of this method is its precision. It produces zero false positives in the test environment; if it flags a segment, a collision is guaranteed to have occurred. However, its main drawback is that it is purely reactive.

Method B: Risk-Based Detection via Bounding Circle and Articulation Angle

The second method follows a proactive and conservative philosophy. Instead of waiting for a collision to occur, it tries to identify maneuvers that are likely to be dangerous based on a combination of contextual factors. A moment is flagged as dangerous only if both of the following conditions are met simultaneously:

1. **Factor 1: Proximity Alert:** This condition checks if the vehicle is in close proximity to an obstacle. The vehicle's footprint is approximated by a single, large bounding circle with radius `radius = (g.Ltr + g.Lh + g.Ltl) / 2` and a center moving between the tractor's front and trailer's rear. Proximity is checked by looking up the distance to the nearest obstacle on a pre-computed Distance Transform Map, a fast operation.

2. **Factor 2: Articulation Alert:** Proximity alone is insufficient. A second trigger is needed to detect when the vehicle is actively turning. An articulation alert is triggered if the absolute hitch angle (calculated using `angdiff`) exceeds a predefined threshold.

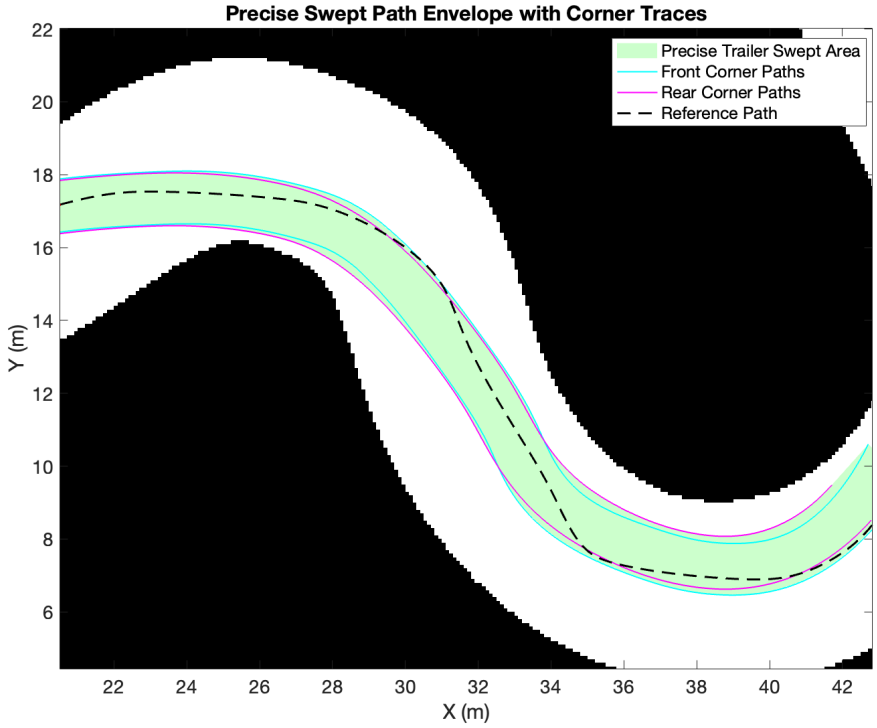


Figure 3.2: The Precise Swept-Path Envelope. The green area represents all space occupied by the trailer during the simulation. The overlap with the black obstacle indicates a collision, triggering the hazard detection.

```
% Logic for the two-factor trigger
hitch_angle = abs(angdiff(truck_sim_orig.theta(i), ...
                          trailer_sim_orig.theta(i)));
articulation_alert = hitch_angle > hitch_angle_threshold;

if proximity_alert && articulation_alert
    is_dangerous(i) = true;
end
```

This two-factor approach is computationally lightweight and can proactively flag tight turns near obstacles, even if no collision occurred. Its primary trade-off is that it is an approximation and may be overly cautious.

3.3 The Maneuver Window: Causal Path Segmentation

A critical insight is that the location of a hazardous event is not the same as the location of its cause. A trailer collision is often the result of a steering action initiated by the tractor much earlier. The "Maneuver Window" algorithm solves this causal mismatch by systematically shifting the start of a detected hazardous segment backward in time and space.

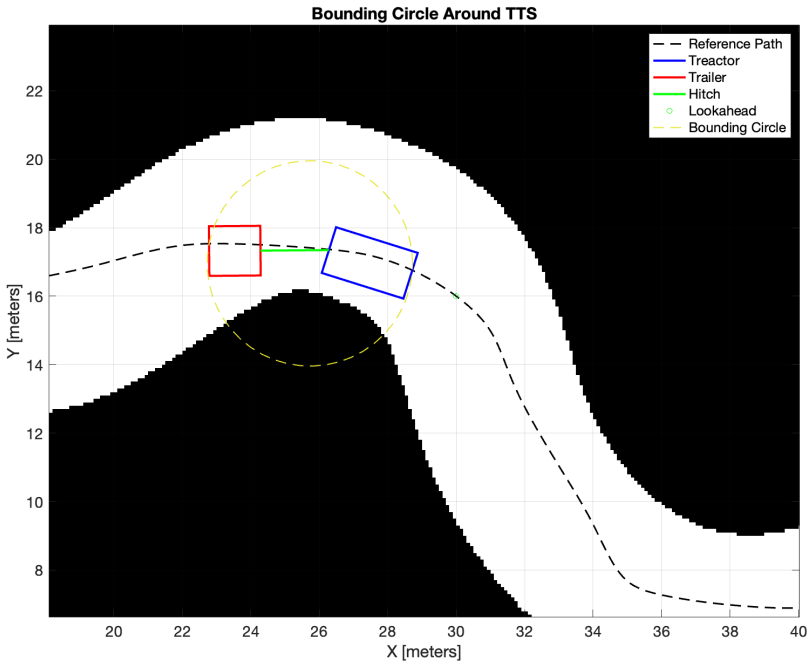


Figure 3.3: The Two-Factor Hazard Trigger. The maneuver is not flagged as dangerous because the vehicle's bounding circle (yellow) is intersecting the inflated obstacle zone, but the hitch angle is within its threshold, indicating straight navigation in a constrained space.

The implementation is a three-step process:

1. **Define Causal Distance:** A physical "causal distance" is defined as the length of the rigid linkage from the tractor's axle to the rearmost point of the trailer.

```
causal_distance = g.Lh + g.Ltl + g.Ltr; % Hitch length + trailer length + Truck
```

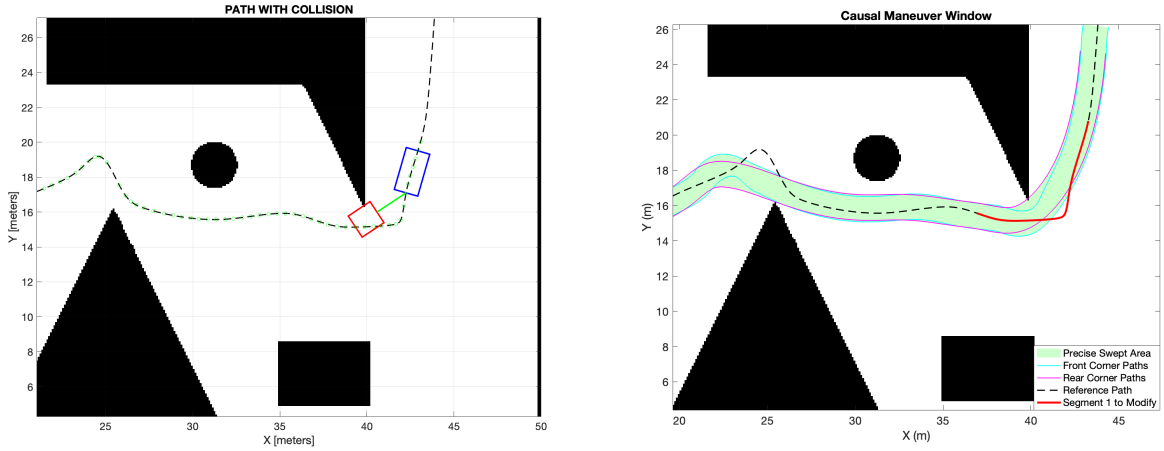
2. **Convert Distance to Index Shift:** This physical distance is translated into a data array index shift by calculating how many array indices correspond to the `causal_distance`.

```
path_distances = vecnorm(diff([x_ref(:), y_ref(:)]), 2, 2);
avg_step_dist = mean(path_distances);
index_shift = round(causal_distance / avg_step_dist);
```

3. **Apply the Window Shift:** The start index of each dangerous segment is shifted backward by the calculated `index_shift`. In contrast, the end index remains unchanged, correctly framing the entire maneuver.

```
% Applying the shift to the raw detected bounds
maneuver_bounds(:,1) = max(1, raw_danger_bounds(:,1) - index_shift);
maneuver_bounds(:,2) = raw_danger_bounds(:,2);
```

The Maneuver Window is a key component for the framework's robustness, as it provides the Bézier curve generator sufficient context and space to find a smooth, safe, and kinematically feasible alternative.



(a) Precise Collision Point.

(b) Causal Maneuver Window in Red

Figure 3.4: The Causal Maneuver Window. Instead of only flagging the path segment where the hazardous event occurs (left image), the algorithm shifts the start point backward by a causal distance, capturing the entire maneuver (right image) that led to the event.

3.4 Informed Replanning and Validation

The final stage involves replacing hazardous segments with safe alternatives. The `modify_curves_bezier` function iteratively searches for the optimal Bézier curve that is certifiably safe according to a custom cost model.

The Iterative Search and Cost-Based Validation

For each hazardous segment, the algorithm explores the space of possible Bézier curves by varying its "handle" parameters d_1 and d_2. For each candidate curve generated:

1. **Simulation:** A fast, localized simulation of the vehicle is run along the candidate Bézier curve.

2. **Safety Evaluation:** The simulated trajectory is then evaluated for safety by the `evaluate_segment_saf` function. This uses `vehicleCostmap` objects to check the cost of the tractor's and trailer's footprint.

3. **Scoring:** A numerical score is assigned based on a weighted sum of several penalties:

    - `COLLISION_PENALTY_ABSOLUTE`: An extremely large penalty for a hard collision.

- `stats_cand_vc.truck_close` and `stats_cand_vc.trailer_close`: A smaller penalty for "close calls" within the inflated costmap region.
- `OUTER_WALL_HIT_PENALTY`: A penalty if the curve itself passes through an obstacle.
- `LOOP_PENALTY`: A penalty if the control points cross, creating a self-intersecting curve.

The algorithm systematically tests curves, seeking to minimize this comprehensive score. The final chosen curve is the one with the lowest score that has a zero collision count. This informed validation ensures that the final modified path is not just geometrically plausible but has been rigorously checked against a sophisticated risk model that accounts for the vehicle's full footprint and a defined safety margin.

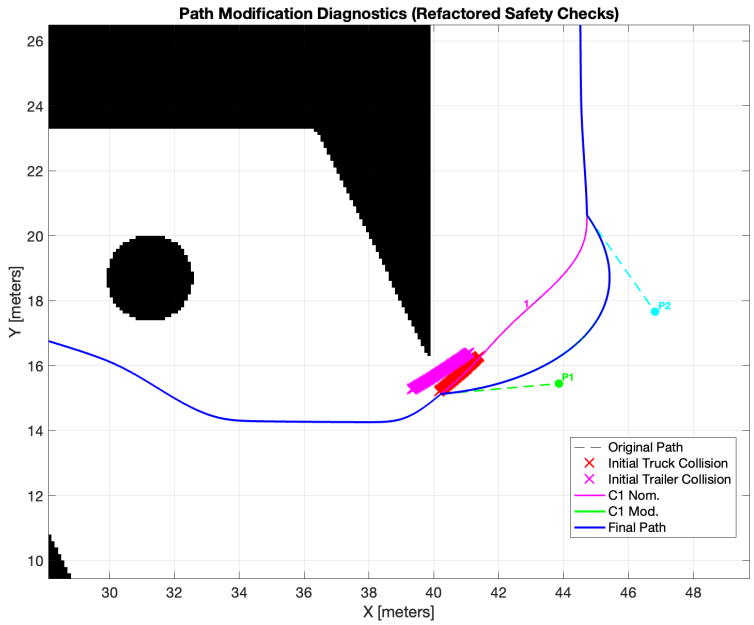


Figure 3.5: Informed Replanning with Bézier Curves. The hazardous nominal segment (magenta) is replaced by an optimized, C^2 continuous Bézier curve (described by points P1 and P2) that is validated against a vehicle costmap to ensure it is collision-free and maintains a safe distance from obstacles.

Chapter 4

Analysis and Discussion of Path Following Control

Generating a path is only part of the autonomous navigation challenge; the vehicle must also be able to execute it accurately and efficiently. This chapter addresses the "Control" component needed to accurately track a trajectory. It focuses on the implementation and analysis of three distinct path tracking controllers, which are responsible for calculating the necessary steering commands to guide the simulated rover along the final, modified trajectory.

The goal of this chapter is to demonstrate how various control strategies and tunings affect tracking performance. This analysis offers critical insights into the practical aspects of such systems, bridging theoretical evaluations with simulation results, where the choice of controller can significantly impact the vehicle's safety and predictability. This concentrated study on the implementation of simple geometric controllers considered three path tracking strategies: a fixed-parameter Pure Pursuit controller, a simple curvature-adaptive Pure Pursuit controller, and a classic heading error-based PID controller. It is of great importance underlining for the reader, that this study on path tracking control accounts only for the tractor module of the system as for the framework's validation a fine-tuned trajectory following is needed for the front module, while will be role of the framework to correct the passive trailer's off-tracking errors and verify that the developed obstacle avoidance module can be plugged into a preexisting robotic tractor and activate when a trailer is present.

4.1 Controller Implementation in the Simulation Environment

The core of the simulation is an iterative process where, at each time step `dt`, the chosen controller calculates an angular velocity ω, which is then used to update the vehicle's state (x, y, θ).

Fixed Lookahead Pure Pursuit

This is the standard implementation. At each iteration, it uses a constant, predefined `fixed_lookahead` distance. The logic, as implemented in the main simulation, is as follows:

1. **Find Target Point:** The vehicle's front axle position is calculated. The `find_lookahead_point` function then searches along the reference path to find the coordinate (t_x, t_y) that is `fixed_lookahead` distance away.

2. **Compute Steering Angle:** The heading error α to this target point is used in the Pure Pursuit formula to calculate the required steering angle δ.

```
alpha = atan2(ty - y, tx - x) - theta;
delta = atan2(2 * wheelbase * sin(alpha), fixed_lookahead);
```

3. **Calculate Angular Velocity:** The steering angle is converted into an angular velocity ω for the bicycle model: $\omega = (v/\text{wheelbase}) \cdot \tan(\delta)$.

Curvature-Adaptive Lookahead Pure Pursuit

This experimental implementation addresses the primary drawback of the fixed controller—its static behavior. The goal is to achieve high accuracy in turns and high stability on straightaways. The logic for this case study modifies the first step of the fixed approach:

1. **Determine Current Lookahead:** The first step for the controller is to identify the closest point on the reference path (`closest_idx`) and the path curvature at that location. It then uses a linear mapping to dynamically calculate the `current_lookahead` for the current time step, clamping it between `min_lookahead` and `max_lookahead`.

```
% Logic for adapting the lookahead distance
lookahead_ratio = min(1.0, current_curvature * curvature_scaling_factor);
current_lookahead = max_lookahead - (max_lookahead - min_lookahead)...
                                                    ... * lookahead_ratio;
```

2. The following steps for calculating δ and ω are identical to the fixed version, but they now use the dynamically adjusted `current_lookahead`.

Heading Error-Based PID Control

As an alternative to geometric methods, a standard Proportional-Integral-Derivative (PID) feedback controller was implemented for comparison. This controller's command is based on the heading error to a target point on the path, not the lateral error. Its goal is to compute an angular velocity that drives this heading error to zero. The implementation follows these steps:

1. **Identify Target Point:** A simple lookahead strategy is used to select a target point (t_x, t_y), a fixed number of indices ahead of the closest point on the path.

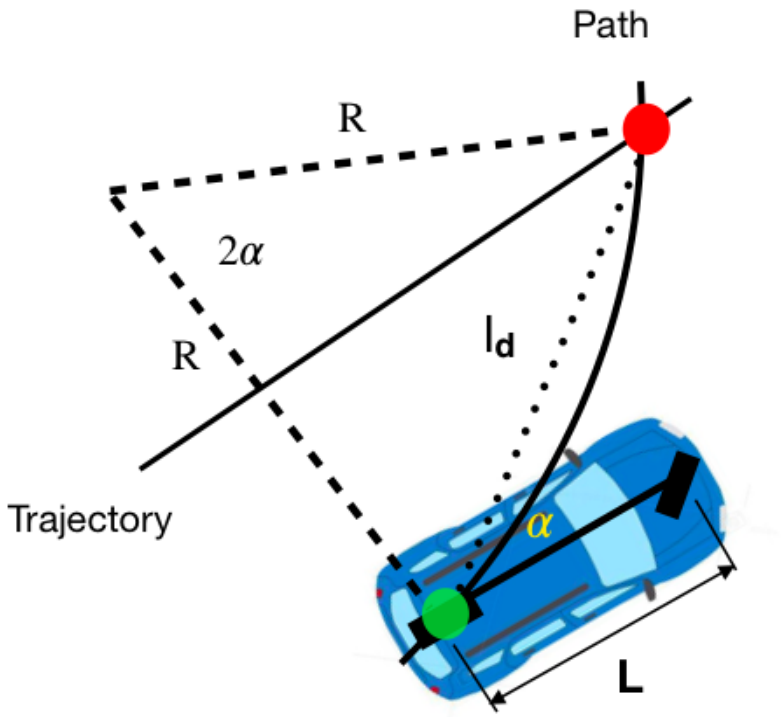


Figure 4.1: The geometric principle of the Pure Pursuit controller. The steering command is calculated to place the vehicle on a circular arc that intercepts the reference path at a target point. Illustration by Ding [17]

.

2. **Calculate Heading Error:** The desired heading (`targetAngle`) to this point is calculated. The heading error `angleError` is the difference between the target heading and the robot's current heading θ.

3. **Compute PID Terms:** The controller calculates the three components of its response based on this heading error:

```
angle_error_sum = angle_error_sum + angleError * dt;
angle_error_rate = (angleError - angle_error_prev) / dt;
P_term = Kp * angleError;
I_term = Ki * angle_error_sum;
D_term = Kd * angle_error_rate;
```

4. **Compute Angular Velocity:** The final angular velocity command ω is the sum of these three terms. The gains K_p, K_i, and K_d are determined automatically using MATLAB's `pidtune` function on a simple integrator plant model, ensuring a stable tuning, while the system is as simple as possible.

4.2 Performance Metrics for Path Tracking Evaluation

The evaluation of controller performance requires specific quantitative performance indicators. Path quality metrics are crucial for "evaluating and comparing the performance of different navigation algorithms" [18]. That's the reason why accuracy, stability, and smoothness are the main goals of a path tracking system. The analysis used lateral error and control effort as essential metrics, which were collected and presented in graphical format:

- **Lateral Error (Cross-Track Error):** serves as the primary tracking accuracy metric because it represents the robot's center (x, y) perpendicular distance from its reference path at any given time. The developed implementation finds the minimum distance and then uses a vector cross product to determine the sign, which is descriptive of whether the robot is to the left or right of the path. Performance has then been evaluated using **Root Mean Square (RMS)** , which provides a measure of the average tracking error over the entire simulation, and **Maximum Absolute Error**, which is critical for safety analysis as it represents the worst-case deviation from the path.

- **Control Effort Smoothness:** The smoothness of the control effort is evaluated by examining the final angular velocity command (ω) over time. A stable controller produces ω values that show continuous patterns, which reduce the risk of mechanical wear and unstable vehicle behavior.

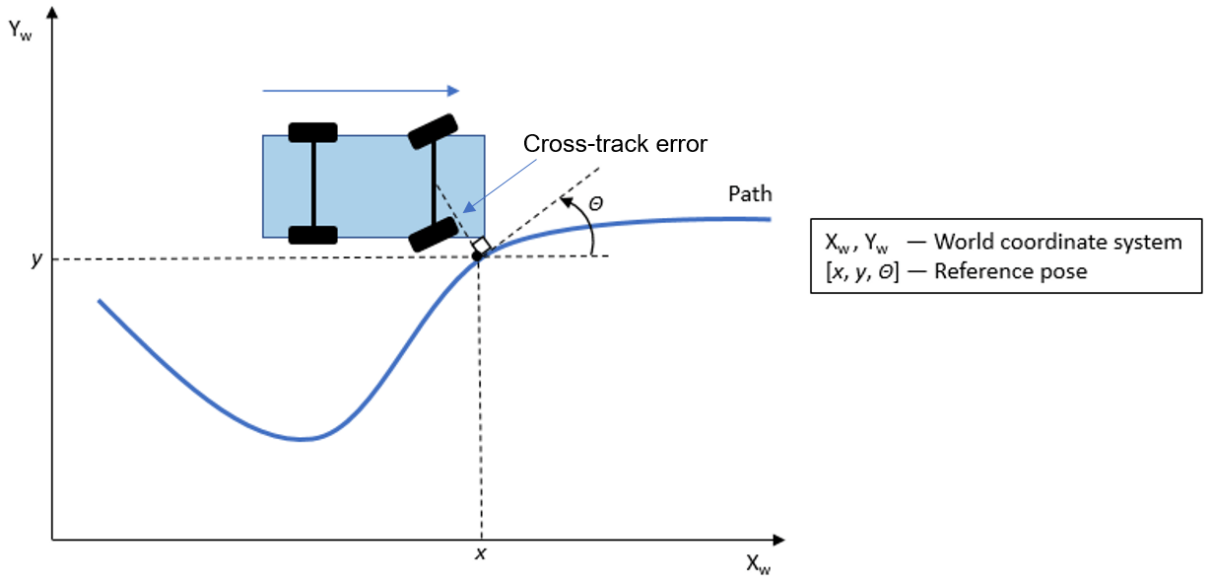


Figure 4.2: Definition of Lateral Error, the primary metric for evaluating path tracking accuracy.

4.3 Experimental Results and Comparative Analysis

A series of experiments will be conducted using a test trajectory with a mix of straight and curved sections to provide a comprehensive comparison of the three controllers.

Controller Performance on Straight vs. Curved Segments

- **On Straight Segments:** The PID controller, because of its integral nature, is expected to demonstrate excellent performance, converging to an almost null steady-state lateral error. The Fixed Pure Pursuit controllers (Both static and adaptive), instead, might present a small but persistent offset. Practically, these characteristics resemble those of these controllers on a step response.

- **On Curved Segments:** During turns, the controllers' behaviors are expected to be different. The Fixed Pure Pursuit controller, independent of its tuning precision, will exhibit a "corner-cutting" characteristic, leading to a large but predictable lateral error that depends on speed and look-ahead distance. The PID controller, attempting to aggressively minimize heading error, may overshoot upon entering and exiting the turn, and obviously struggles more as curves get tighter. The Adaptive Pure Pursuit controller is hypothesized to offer the best compromise: by shortening its lookahead distance in response to high curvature, it should track the curve more accurately than its fixed counterpart but more smoothly and with less overshoot than the PID controller.

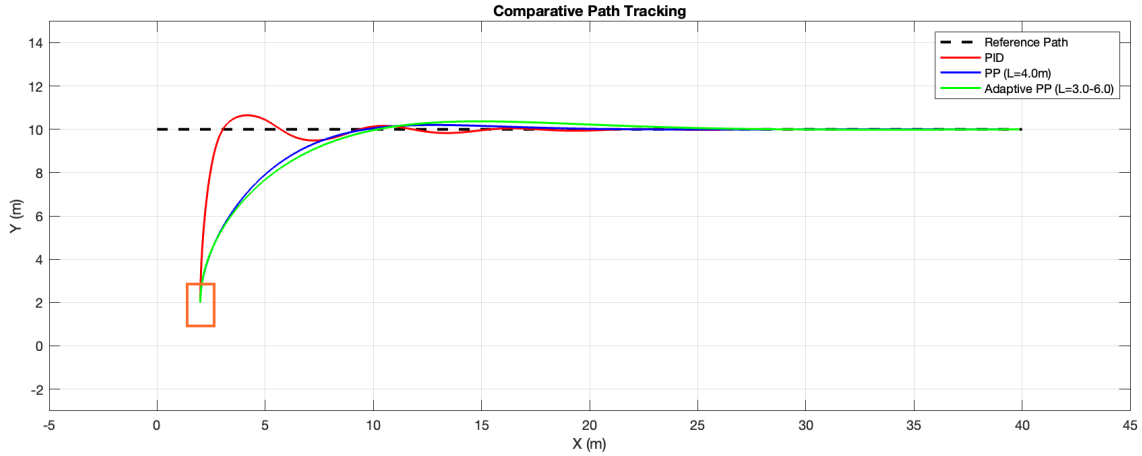


Figure 4.3: A plot showing the reference path and the simulated trajectories for all three controllers attempting to track a straight path.

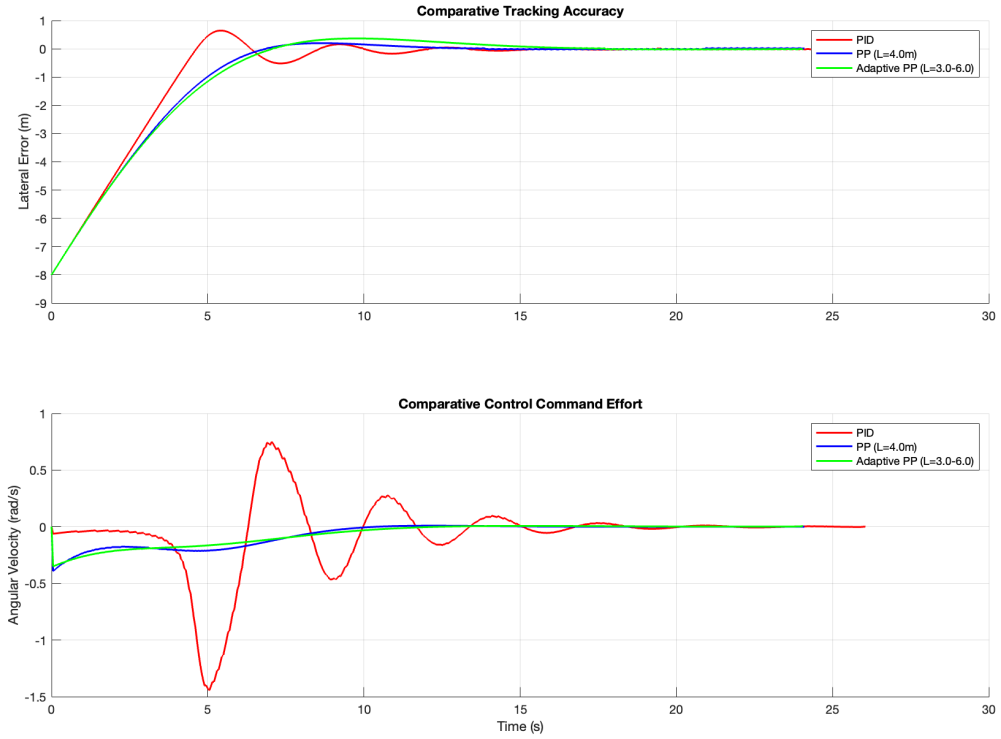


Figure 4.4: A plot of Lateral Error vs. Time attempting to track a straight path

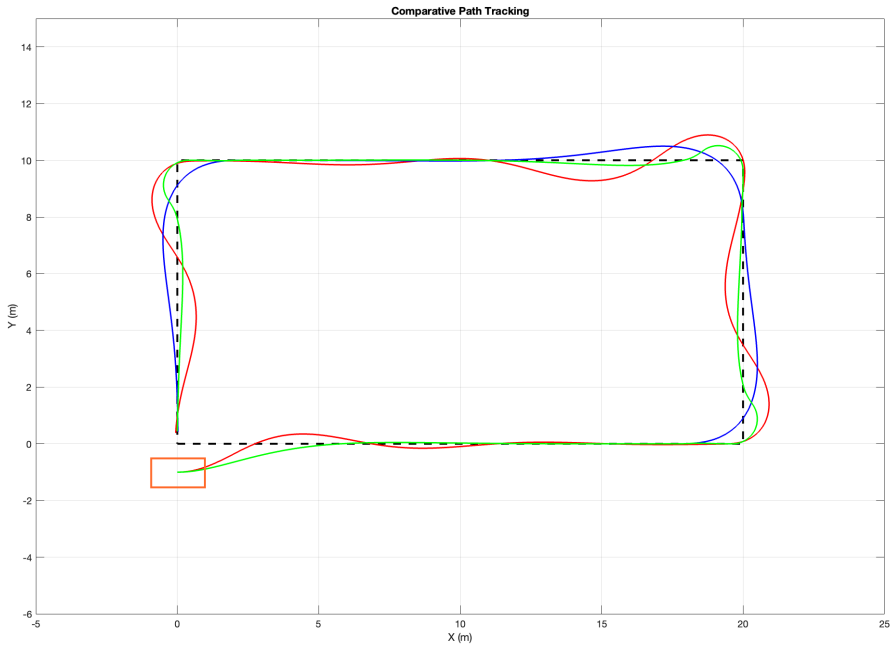


Figure 4.5: A plot showing the reference path and the simulated trajectories for all three controllers on a curved path. The corner-cutting of the Fixed Pure Pursuit, the potentially tighter tracking of the Adaptive Pure Pursuit, and the overshoot of the PID controller are visible in this image.

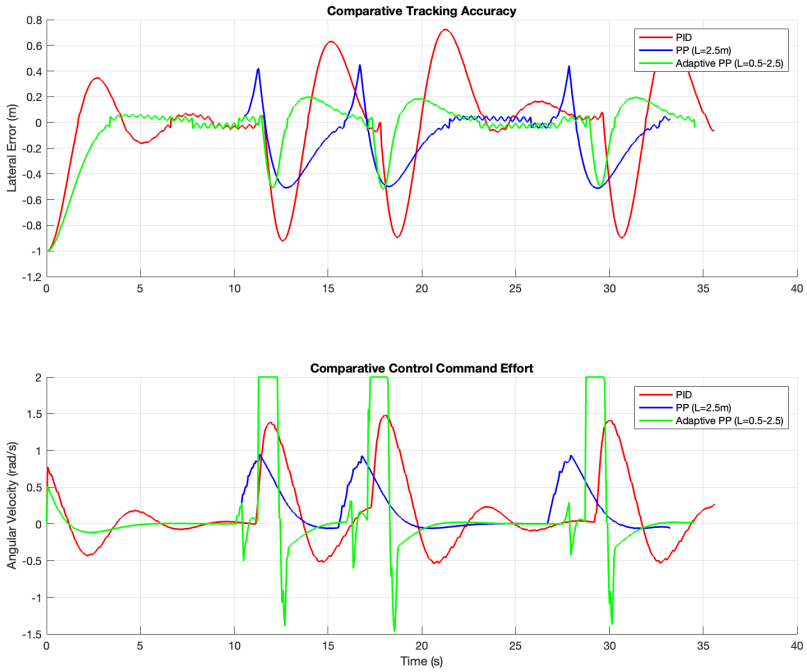


Figure 4.6: A plot of Lateral Error vs. Time attempting to track a curved path

4.4 Parametric Analysis: The Influence of Lookahead Distance

The performance of the Pure Pursuit controller is critically dependent on the lookahead distance. This parameter must be accordingly tuned as it directly impacts tracking accuracy and control stability. A detailed parametric analysis and simulation of the Fixed Pure Pursuit controller was conducted to illustrate this intrinsic relationship. The simulation will be run multiple times with different tuning values for `fixed_lookahead` (e.g., a short value, a fine-tuned value, and a long value).

- **Small Lookahead Distance:** A short lookahead forces the controller to be highly reactive. The robotic system will experience aggressive tracking that may become oscillatory, showcasing how the vehicle "weaves" across the reference path. This will be visible as high-frequency oscillations in both the lateral error and control command plots.
- **Large Lookahead Distance:** A long lookahead has a significant damping effect. The result is very smooth and stable control. However, the overdamped nature of this controller tuning makes the path tracking highly inaccurate, as we experience the tractor "cutting corners," leading to large lateral errors in turns.

"The lookahead distance determines how aggressively the vehicle will attempt to track the path" [7]. The following plots clearly demonstrate this trade-off. On the one hand These fixed charac-

teristics of the Pure Pursuit Controller are the strengths that make it predictable, and preferable for general scenarios that don't necessitate perfect curve tracking, but on the other hand serves to highlight the motivation for the adaptive algorithm, which is designed to select the "best" lookahead distance for every point along the path, rather than being limited by a single, global compromise.

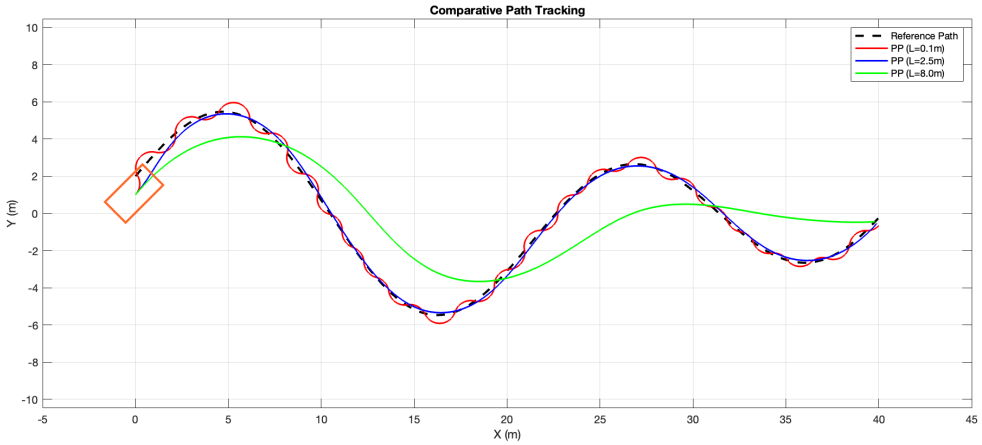


Figure 4.7: A plot showing the reference path and three simulated trajectories for the Fixed Pure Pursuit controller, each with a different lookahead distance. The plot clearly shows the small-lookahead path oscillating and the large-lookahead path cutting the corner.

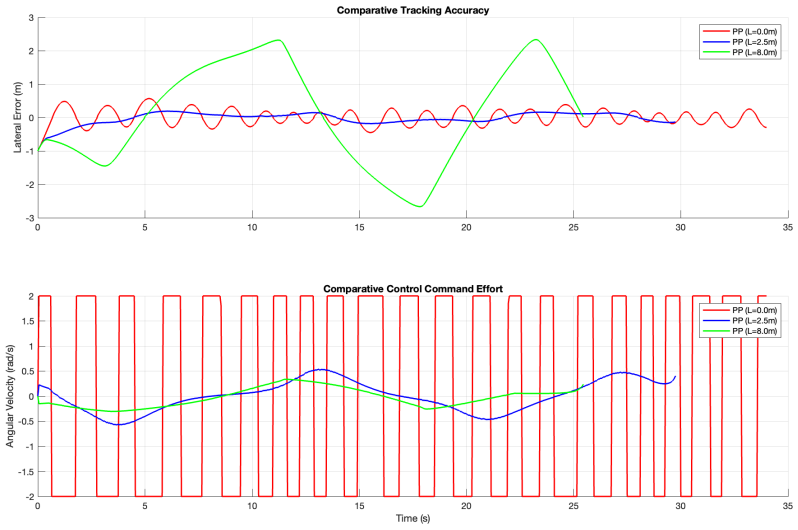


Figure 4.8: A plot of Lateral Error vs. Time, showing the error curves for the three different lookahead distances, illustrating the trade-off between oscillation and large, smooth corner-cutting error.

Chapter 5

Results and System Validation

This chapter presents the experimental validation of the complete offline path modification and obstacle avoidance system. The objective is to demonstrate the effectiveness and the characteristics of the proposed hazard detection strategies through a set of simulations conducted in an environment purposely designed to stress the system with diverse characteristics.

This chapter will study the entire "Plan -> Detect -> Modify -> Validate" cycle. The results will be compared to evaluate the trade-offs between the two methods in terms of path safety, efficiency, and overall behavior, finally validating the framework's success in transforming kinematically naive paths into trajectories that are safe for an articulated agricultural rover.

The efficacy of the previously described hazard detection methods is validated through different simulations, considering a baseline naive path with guaranteed collision. To ensure a direct comparison among the previously described hazard detection methods, the exact same setup conditions were provided

The tests were conducted in a complex environment containing static obstacles, narrow passages, and sharp corners. Moreover, the start and end points of the mission were chosen to require at least two turns greater than 90 degrees to reach the destination. This was done to produce significant off-tracking and create a challenging test case that could be feasible by a standard robotic autonomous tractor, but would fail to drive an articulated system safely.

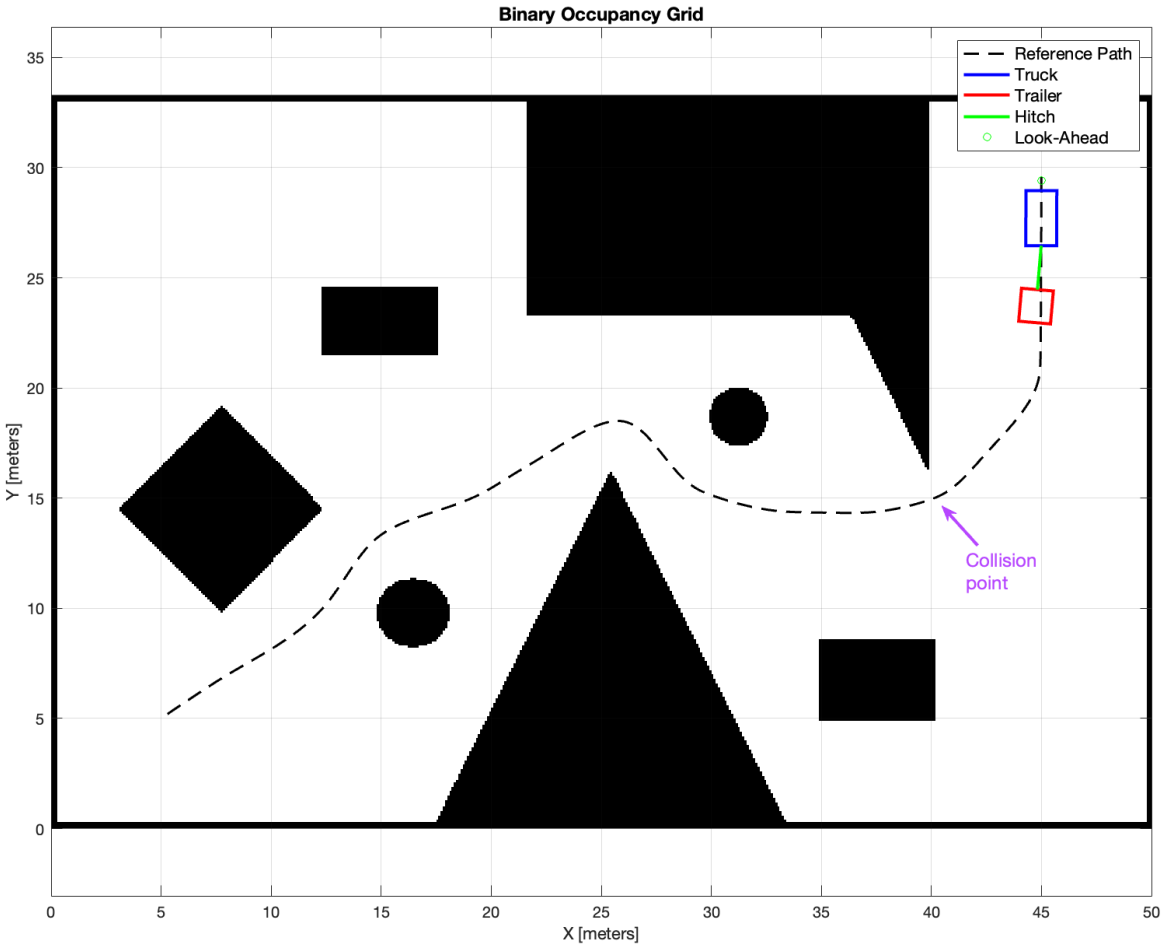


Figure 5.1: The initial smoothed reference path for the test scenario. While geometrically continuous and collision-free for a point-mass, its safety for the articulated rover is unverified.

5.1 Case Study 1: Path Modification using the Envelope Approach (Method A)

Hazard Detection

Following the setup, the first phase of the process is to run a preliminary simulation of the rover along the calculated reference path. The results of this simulation can be easily visualized through the plotted path envelope of the trailed body. As previously anticipated, the simulation reveals a collision. The green, semi-transparent area, which represents the precise swept-path envelope of the trailer, clearly shows how the trailer's footprint overlaps with the black portion of the map - the obstacle.

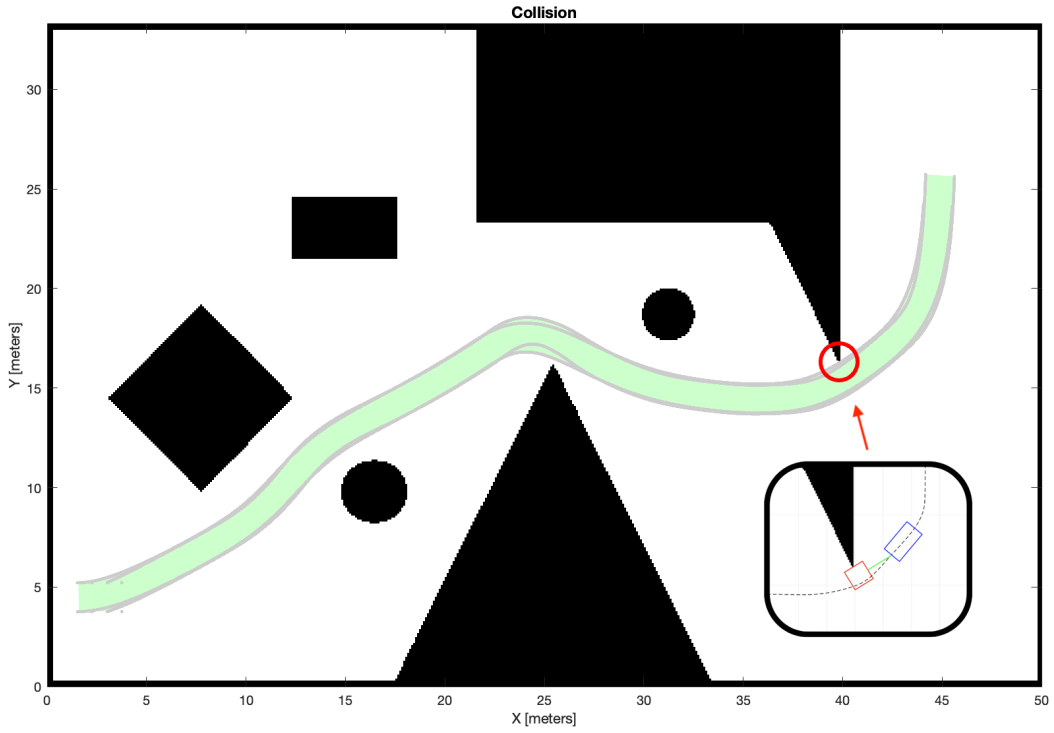


Figure 5.2: Collision detection via the Precise Swept-Path Envelope. The green area shows the trailer's true swept path, with the overlap and red markers indicating an unambiguous collision during the tight turn.

Identification and Modification

After the collisions are identified, the Maneuver Window is calculated. The start of the hazardous segment is then shifted backward along the path from the point of first collision to frame the entire problematic maneuver and act at the true origin of it. Figure 5.3 shows the original reference path with the identified maneuver window segment highlighted in red. This segment is then passed to the `modify_curves_bezier` function, which performs an informed search

for a safe, C² continuous Bézier curve to replace it and stitch a feasible path at its place. The resulting modified path, featuring a wider, safer arc, is shown in Figure 5.4.

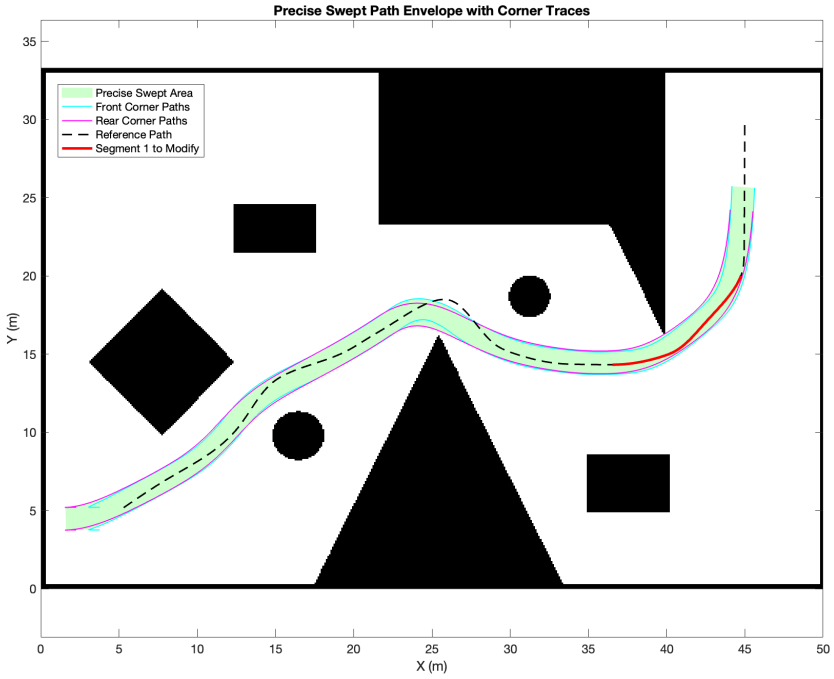


Figure 5.3: The identified hazardous segment for modification. The Maneuver Window algorithm has correctly framed the entire maneuver leading to the collision.

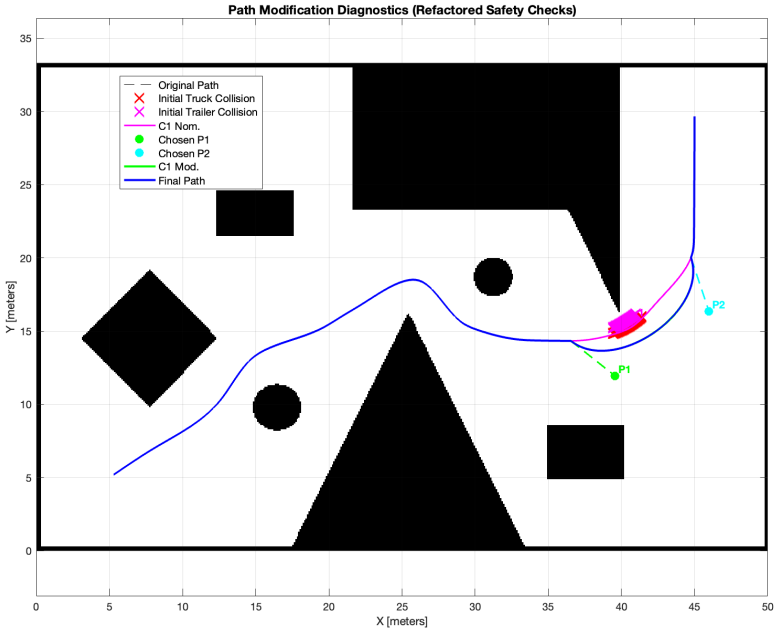


Figure 5.4: The modified trajectory. The hazardous segment has been replaced by an optimized Bézier curve that provides more clearance.

Validation

The final step is to validate the new, modified path. A full simulation of the rover is executed along this new trajectory. Figure 5.5 shows the swept-path envelope of the trailer from this final validation run. The result is a clear success: the new envelope is fully contained within the free space of the corridor, with no overlaps with any obstacles. The vehicle successfully navigates the turn without collision.

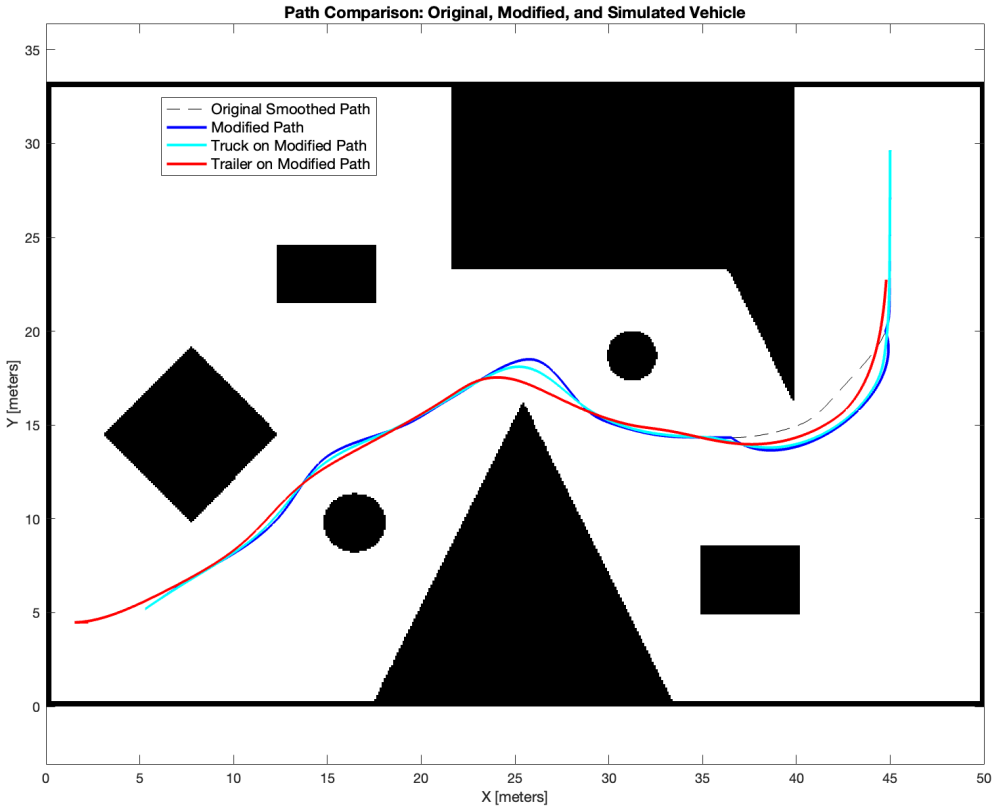


Figure 5.5: Successful validation of the modified path. The new paths tracked by truck and trailer confirm the effectiveness of the modification.

5.2 Case Study 2: Path Modification using the Bounding Circle Approach (Method B)

Hazard Detection

Keeping the setup conditions unmodified, a simulation is now conducted with the second proposed detection method. This time, the data is analyzed by the Bounding Circle and Hitch Angle module. Unlike Method A, which required an actual collision, Method B flags moments of high risk. The system is triggered when experiencing simultaneous proximity and high articulation

when turning (it can also be a collision). Figure 5.6 shows a snapshot of the simulation from this detection phase. The purple bounding circle indicates the points where the `is_dangerous` flag was set to true. Even in a hazardous segment with no collision, the system proactively identifies the maneuver as being kinematically stressful and occurring in a constrained space, thus classifying it as dangerous.

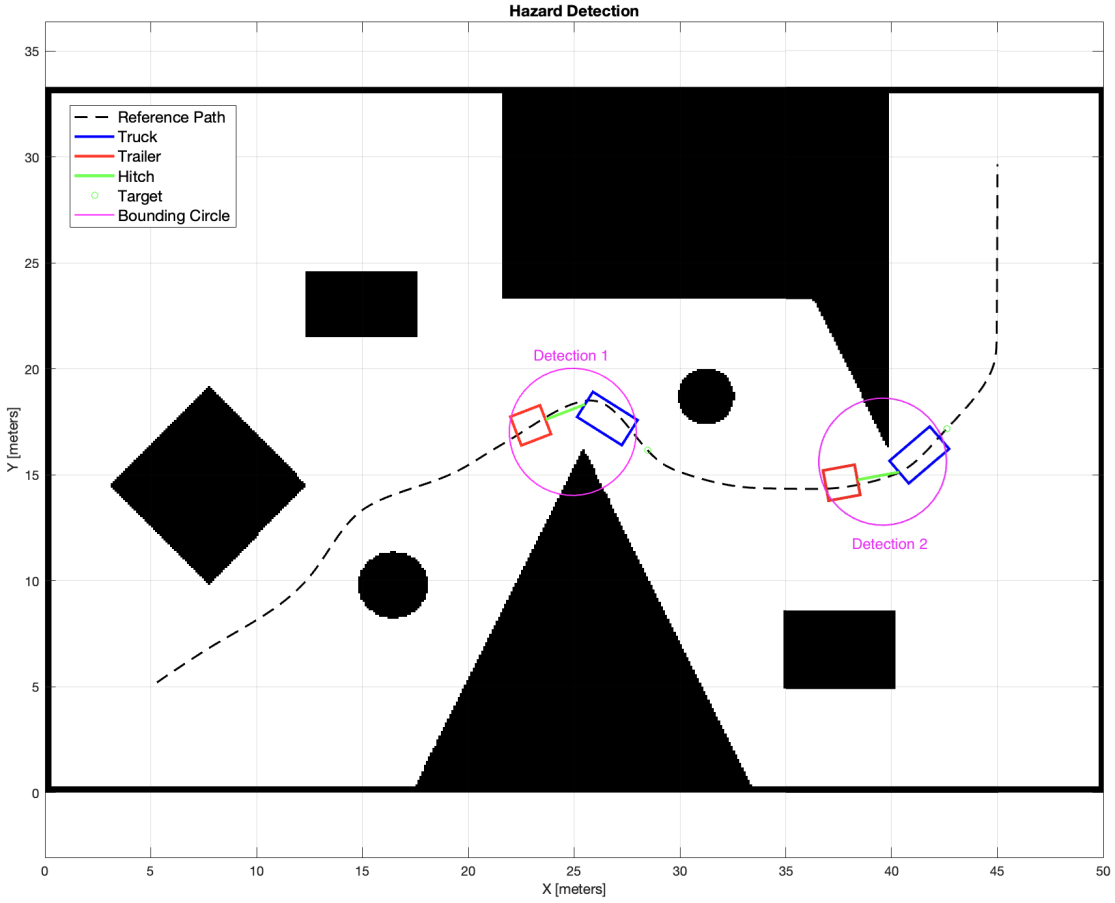


Figure 5.6: Risk detection via the Bounding Circle and Hitch Angle. The purple color highlights points where the vehicle was simultaneously too close to an obstacle and in a state of high articulation, flagging the maneuver as high-risk.

Identification and Modification

As with Method A, the Maneuver Window is applied to the segments flagged by the `is_dangerous` vector. This again ensures that the entire maneuver is captured for modification. The resulting segment is then passed to the `modify_curves_bezier` function. The function generates a safe Bézier curve replacement, resulting in the final modified path for this method, as shown in Figure 5.7.

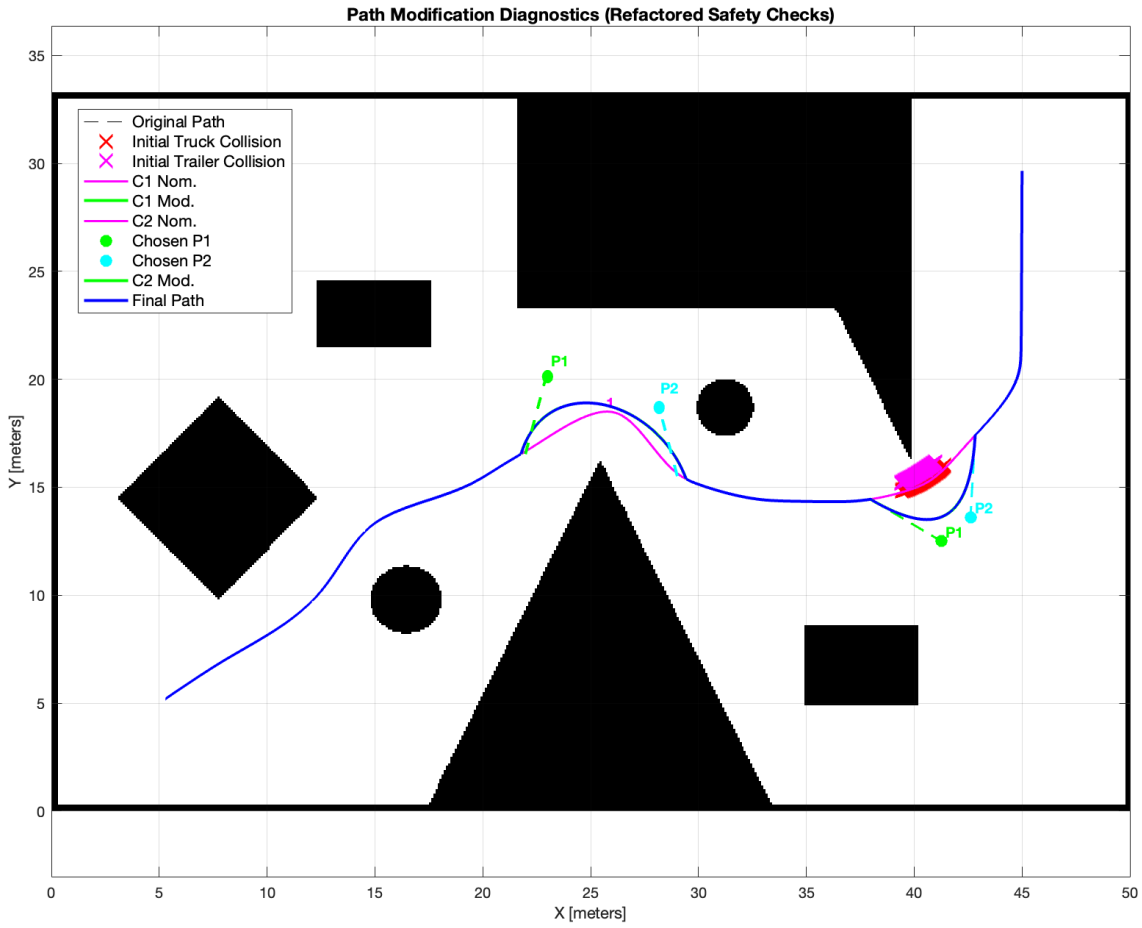


Figure 5.7: The final modified path generated based on Method B's detection.

Validation

The final validation simulation is finally run on the path modified by Method B. If the system manages to successfully modify the path, and the background check executed by the framework detects no collisions, the path is safe for navigation of an articulated vehicle. Figure 5.8 shows a snapshot from this final path in the proposed environment. The reader can easily identify the executed modifications, thus validating the system. The bounding circle may still signal some danger due to the insidious nature of the path, but since the trajectory was previously validated, we are confident in the soundness of the simulation.

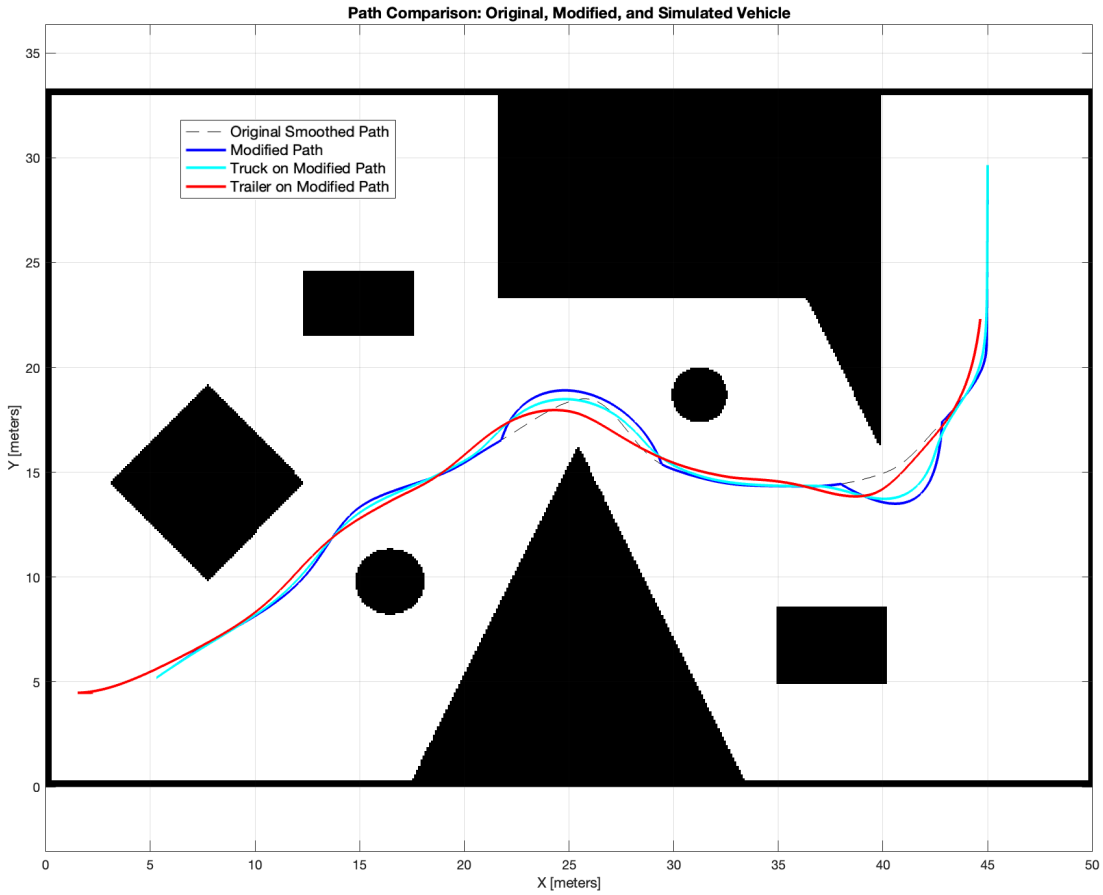


Figure 5.8: Successful validation of the risk-based modification. The new simulation shows no hard collisions, confirming that the hazardous portions of the path have been successfully mitigated by the new path.

5.3 Discussion and Comparison of Final Trajectories

The best direct way to visually compare the outcomes is to overlay the two final modified paths on the same map. Figure 5.9 shows the original naive path, the path modified by the envelope method, and the path modified by the bounding circle method.

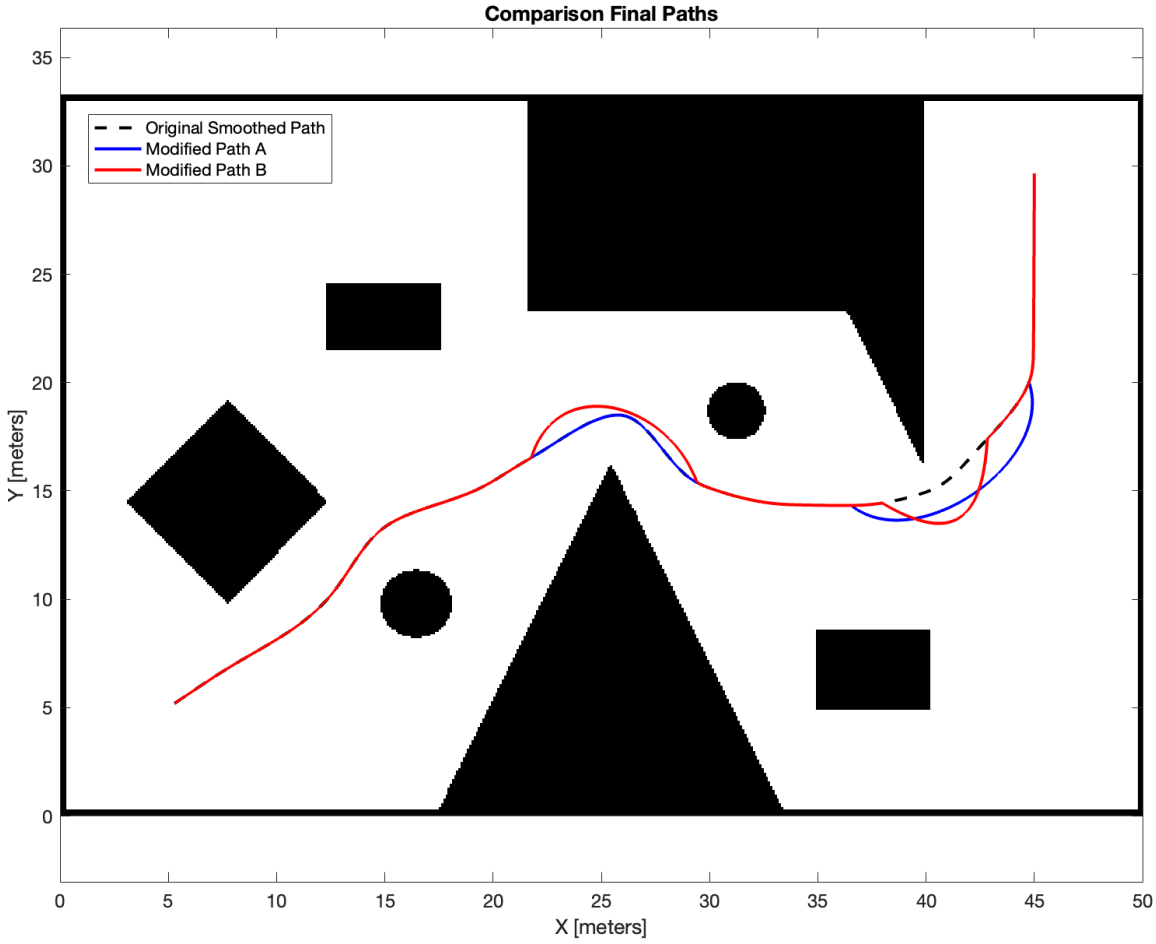


Figure 5.9: Comparative overlay of the final trajectories. The path generated by the proactive Method B (red) takes a wider, more cautious approach than the path from the reactive Method A (blue).

Qualitative Analysis

The visual comparison immediately reveals the fundamental difference in the philosophies of the two methods.

- **Method A (Envelope):** The resulting path is more "aggressive" or efficient, but for a real-world implementation, an ulterior safety buffer might be needed. It modifies the original trajectory only as much as is strictly necessary to resolve the collision. The curve is widened just enough to provide clearance for the trailer's body, resulting in a path that

is safe but potentially has a smaller safety margin, which we could theoretically increase by inflating the map. It is a minimalist correction that solves a known problem.

- **Method B (Bounding Circle):** The resulting path is visibly more "cautious" and conservative, but this could lead to other criticalities. As this method is triggered by the risk of a tight turn in a confined space, it tends to initiate the turn earlier and create a wider, gentler arc, at the cost of risking modifying non-colliding portions of the path. This approach prioritizes maximizing the safety margin over minimizing the deviation from the original path. Due to its architecture, it might fail in an overly tight environment, as the whole path might be flagged as dangerous.

Quantitative Analysis

To move beyond qualitative assessment, we can compare objective metrics derived from the final validation simulations without tweaking any parameters to capture the raw response of the modification systems. For this comparison, we focus on two critical performance indicators: the efficiency of the final path, measured by its total length, and the minimum clearance maintained from obstacles. As noted by Lattarulo et al. [18], "these metrics are fundamental for evaluating the quality of trajectories for autonomous vehicles".

Path Efficiency: Length and Deviation

A primary goal of any path planner is efficiency. In this context, the total path length serves as a direct proxy for operational efficiency, as a shorter path typically corresponds to lower travel time and energy consumption. An ideal modification algorithm should ensure safety while minimizing the deviation from the original, shorter path.

The data in Table 5.1 provides a clear comparison of the path lengths resulting from each modification method. The original, naive path generated by the PRM and smoothing workflow was 57.30 meters.

Table 5.1: Quantitative comparison of the final path lengths.

Method	**Original Path Length**	**Modified Path Length**	**Path Length Increase (%)**
Envelope	57.30 m	58.92 m	**2.84 %**
Bounding Circle	57.30 m	60.11 m	**4.91 %**

The results confirm that both methods necessarily increase the path length to create safer, wider turns. However, the degree of this modification is significantly different between the two.

- **Method A (Envelope)**, being reactive, provides a minimalist correction. It increases the path length by only **2.84%**. This is because its detection is triggered by a guaranteed

collision. The ‘Maneuver Window‘ identifies the smallest possible segment that contains the causal maneuver, and the path gets altered just enough to resolve this specific collision. The result is a highly efficient path that stays as true as possible to the original plan.

- **Method B (Bounding Circle)**, being proactive, is more conservative. It increases the path length by **4.91%**, nearly double the increase of Method A. Its two-factor trigger (proximity + articulation) flags a larger region as "high-risk," even before a collision is guaranteed. This approach prioritizes reducing kinematic stress and potential risk over minimizing path length.

Safety Margin: Minimum Obstacle Clearance

While path length measures efficiency, the minimum clearance from obstacles is a direct measure of the trajectory’s safety. A larger safety margin makes the path more tolerant to real-world uncertainties such as controller tracking errors, wheel slip, and odometry inaccuracies.

Figures 5.10 and 5.11 show the time-series data of the minimum clearance for both the tractor (blue) and trailer (red) throughout the final validation simulations for each method.

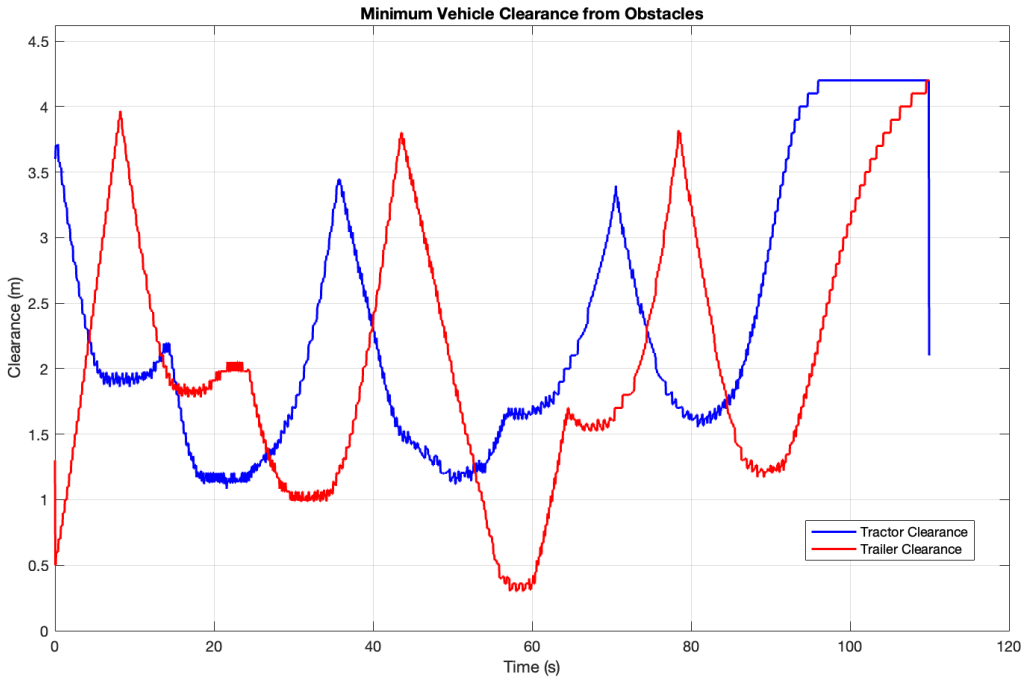


Figure 5.10: Vehicle clearance throughout the simulation on the path modified by **Method A (Envelope)**. The trailer clearance (red) is the critical factor, touching a minimal value lower than $0.3m$ in the tightest turns.

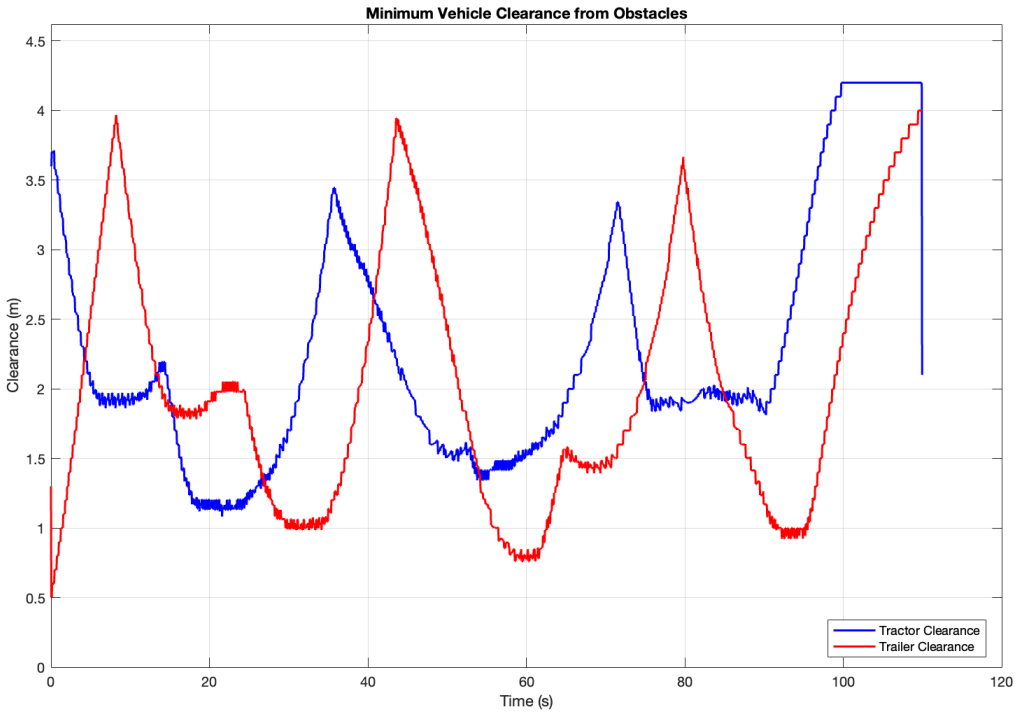


Figure 5.11: Vehicle clearance throughout the simulation on the path modified by **Method B (Bounding Circle)**. The proactive modification results in a substantially larger minimum trailer clearance throughout the maneuver.

A detailed analysis of these plots yields several key insights:

1. **Trailer as the Critical Component:** In both simulations, the troughs of the plots (the points of minimum clearance) occur during the turns. Furthermore, the trailer's clearance (red line) is consistently the limiting factor, dipping significantly lower than the tractor's clearance. This visually confirms that the off-tracking phenomenon is the primary source of collision risk, validating the thesis's core problem statement.

2. **Method A (Reactive) - Sufficient but Minimal Clearance:** Figure 5.10 shows that Method A successfully achieves its goal: the minimum trailer clearance never reaches zero, proving the collision has been resolved. However, the safety margin is critically small, with the clearance dipping to **approximately 0.3 meters** at its lowest point (around the 60-second mark). This result is a direct reflection of its philosophy: it solves the problem precisely as it was defined, but does no more. The path is safe, but not necessarily robust.

3. **Method B (Proactive) - Maximized Safety Margin:** In contrast, Figure 5.11 demonstrates the power of the risk-based approach. The overall shape of the clearance profile is similar, but the troughs are significantly higher. The minimum trailer clearance for the path generated by Method B is **approximately 0.8 meters**, more than double that of Method A (the origin of the simulation at $t = 0$ shall not be considered). By flagging

the maneuver as a "risk" rather than a "failure," the system attempts to generate a path that is organically safer and more conservative, providing a much larger buffer against unforeseen disturbances.

In summary, the quantitative data present a clear and compelling trade-off. Method A produces a more efficient path by performing a minimal, targeted correction. Method B, while slightly less efficient in terms of path length, delivers a far more robust and safe trajectory by proactively maximizing the clearance during high-risk maneuvers.

5.4 Overall System Performance Evaluation

The success of this thesis lies not only in the development of two specific algorithms, but also in the validation of the entire "Plan -> Detect -> Modify -> Validate" paradigm as a robust and viable strategy for safe autonomous navigation of articulated vehicles.

Success of the Framework Architecture

The results confirm that the proposed offline modification framework is a success. This layered approach can also be viewed as "an instantiation of a broader three-layer architecture, where a planner generates a solution, a sequencing or executive layer (our framework) refines and validates it, and a reactive controller executes it" [19]. The success of the case studies validates that this offline refinement stage is a powerful method for securing a generic plan against the complex kinematics of an articulated vehicle.

The Critical Role of Informed Replanning (The "Background Check")

A crucial aspect of the framework's success is the intelligence embedded within the modification module itself. The system's robustness does not stem solely from the detection of hazards, but equally from the informed replanning process carried out by the `modify_curves_bezier` function. This function performs its own high-fidelity "background check" on every candidate Bézier curve, testing an array of possible solutions and choosing the fittest one using the `vehicleCostmap`. This internal validation ensures that the final modified curve is not only different but also safer. For instance, a segment flagged as "high-risk" by Method B might not be modified if the high-fidelity `vehicleCostmap` validation determines the vehicle's footprint maintains an acceptable clearance. Demonstrating a robust, multi-stage validation process.

Synthesis of Methodological Pros and Cons

The two case studies have clearly illuminated the distinct characteristics and trade-offs of the reactive and proactive detection methods. These findings are summarized below.

Table 5.2: Summary of the advantages and disadvantages of the two hazard detection methodologies.

Feature	**Method A (Envelope-Based)**	**Method B (Circle-Based)**
Philosophy	Reactive. Corrects guaranteed, simulated failures.	Proactive. Prevents potential high-risk situations.
Precision	Geometrically Exact. Zero false positives.	Conservative Approximation. May flag maneuvers that are technically safe.
Path Efficiency	Higher. Modifies the path minimally, resulting in shorter trajectories.	Lower. Tends to create wider, gentler curves, resulting in slightly longer paths.
Safety Margin	Sufficient. Guarantees collision avoidance but may result in smaller clearances.	Maximized. Prioritizes a large safety margin and greater obstacle clearance.
Use Case	Ideal for environments where path optimality is critical.	Ideal for safety-critical applications where maximizing robustness is the primary goal.

Chapter 6

Conclusions and Future Work

This thesis was initiated to address the profound challenge of reconciling the abstract domain of geometric path planning with the complex physical reality of an articulated agricultural rover. While previous work has addressed this problem in structured environments, this research ventured into the more demanding domain of unstructured open-field navigation. This final chapter presents a summary of the work accomplished, a critical evaluation of its limitations, and an outlook on promising avenues for future research that this project has unveiled.

6.1 Summary of Work and Key Contributions

The fundamental problem investigated was not the absence of a path, but the absence of a safe one. Standard planning algorithms, such as the Probabilistic Roadmap, generate trajectories that disregard the complex kinematics of a tractor-trailer system, which can lead to collisions caused by the off-tracking phenomenon. This thesis presents the successful design, implementation, and validation of a comprehensive offline framework to resolve this issue.

The primary contribution of this work is a complete system that analyzes a kinematically naive reference path, deterministically identifies its hazardous segments, and intelligently modifies them using Bézier curves to produce a final trajectory that is both safe and feasible. The key contributions of this work are as follows:

- **A Complete Simulation Environment:** A robust testbed was developed in MATLAB, serving as a virtual laboratory for the entire lifecycle of planning, modification, and control for the articulated rover.
- **A Comparative Study of Novel Hazard Detection Methods:** Two distinct, deterministic methods for hazard identification were developed and compared. The first, a collision-based Envelope Approach, offers geometric precision by reacting to guaranteed collisions. The second, a risk-based Bounding Circle Approach, provides a proactive and conservative assessment by identifying potentially dangerous maneuvers before a collision occurs.
- **The "Maneveur Window" Logic:** A deterministic method was developed to correctly link the effect (a trailer collision) to its cause (an earlier tractor maneuver). This logic

ensures that the path modification module consistently acts on the correct segment of the path.

- **An Informed Replanning Module:** The framework utilizes Bézier curves within an intelligent replanning system. The `modify_curves_bezier` module performs an internal validation using high-fidelity `vehicleCostmap` objects, ensuring that any modification is a verifiable improvement.

- **A Holistic "Planning-to-Control" Analysis:** The work was extended beyond path generation to include a comparative analysis of path following controllers, thereby closing the loop between planning and execution by evaluating the tracking accuracy of the final trajectories.

6.2 Limitations of the Current System

A critical evaluation of this research requires the acknowledgment of its boundaries. While the developed framework represents a significant step forward, its limitations illuminate the path for future work.

The most significant limitation is the framework's offline nature. The entire process of detection and modification occurs before the mission begins. Consequently, the system is incapable of reacting to dynamic or unforeseen obstacles and operates on the assumption of a static, known environment.

Furthermore, a "sim-to-real gap" must be considered. The kinematic model used, while effective for this study, is an idealization. It does not account for real-world complexities such as wheel slip, terrain irregularities that affect vehicle dynamics, or the noise and inaccuracies of physical sensors. The performance of the path tracking controllers, for instance, relies on a perfect state estimation that would be a significant challenge to achieve in a practical application.

Finally, the implemented solutions, while deterministic, are based on heuristics. The placement of Bézier control points and the tuning of controller parameters are not derived from a formal optimization process. Consequently, while the resulting path is safe, it is not guaranteed to be optimal with respect to metrics such as time, energy consumption, or path length.

6.3 Avenues for Future Research and Development

The identified limitations provide a clear roadmap for future research. This project serves as a solid foundation upon which more advanced and robust systems can be built.

Towards Advanced Control and Online Replanning

A logical extension of this work is to move beyond offline modification and towards real-time, reactive control. A promising direction is the implementation of Model Predictive Control (MPC). An MPC controller could use the vehicle's kinematic model to predict its future trajectory over a short horizon and continuously optimize steering commands to follow a reference path while actively avoiding obstacles and respecting physical constraints. This would elegantly merge the planning and control layers into a single, powerful, and reactive system.

Bridging the Sim-to-Real Gap

The practical implementation of this system would require addressing several challenges. The first is hardware deployment on a physical agricultural rover, which would necessitate the integration of a full sensor suite (GPS, IMU, LiDAR, etc.). The challenge of state estimation would become paramount, likely requiring sensor fusion algorithms such as the Extended Kalman Filter (EKF) to provide a robust estimate of the vehicle's pose.

Expanding the Arsenal of Techniques

The framework itself can be enhanced. One could envision a hybrid planning system that uses the computationally fast Bounding Circle method as a first-pass risk assessor, triggering a more detailed, high-fidelity check only when a high-risk maneuver is detected. Furthermore, the deterministic models could be augmented with machine learning approaches, such as training a neural network to predict trailer off-tracking or to learn complex replanning strategies.

Finally, the scope could be broadened to more complex Systems of Systems. The obstacle avoidance strategies could be scaled to a fleet of cooperating rovers or to a more complex articulated vehicle with N-trailers (SNTs). The principles of proactive hazard detection and informed replanning developed in this thesis provide a valuable starting point for tackling these future challenges in autonomous agricultural robotics.

Bibliography

[1] Eleonora Moreno. Strategie di guida autonoma in retromarcia per robot mobili articolati = Reverse autonomous driving strategies for articulated mobile robots. Tesi di laurea magistrale in ingegneria meccanica, Politecnico di Torino, 2021. Supervisors: Giuseppe Quaglia, Andrea Botta.

[2] Roland Siegwart, Illah R. Nourbakhsh, and Davide Scaramuzza. *Introduction to Autonomous Mobile Robots*. MIT Press, 2011.

[3] A. M. Kamaruzaman and M. J. E. Salami. A review of vision-based navigation for autonomous agricultural vehicles. *Journal of Terramechanics*, 77:1–16, 2018.

[4] Steven M. LaValle. *Planning Algorithms*. Cambridge University Press, 2006.

[5] Mohamed Elbanhawi and Milan Simic. Sampling-based robot motion planning: A review. *IEEE Access*, 2:56–77, 2014.

[6] Lydia E. Kavraki, Petr Svestka, Jean-Claude Latombe, and Mark H. Overmars. Probabilistic roadmaps for path planning in high-dimensional configuration spaces. *IEEE Transactions on Robotics and Automation*, 12(4):566–580, 1996.

[7] Jarrod M. Snider. Automatic steering methods for autonomous automobile path tracking. Technical Report CMU-RI-TR-09-08, Carnegie Mellon University, The Robotics Institute, 2009.

[8] The MathWorks, Inc. Pure Pursuit Controller. MATLAB & Simulink Navigation Toolbox Documentation, 2025.

[9] Eduardo F. Camacho and Carlos Bordons. *Model Predictive Control*. Springer, 2007.

[10] Juha Backman, Timo Oksanen, and Arto Visala. Model predictive control of an agricultural tractor-trailer system: Path tracking and jackknife avoidance. *Biosystems Engineering*, 121:88–102, 2014.

[11] The MathWorks Inc. MATLAB version r2024a. Natick, Massachusetts, 2024.

[12] Sebastian Thrun, Wolfram Burgard, and Dieter Fox. *Probabilistic Robotics*. MIT Press, 2005.

[13] James D. Foley, Andries van Dam, Steven K. Feiner, and John F. Hughes. *Computer Graphics: Principles and Practice*. Addison-Wesley, 1990.

[14] Sanghyeon Park, Jihong Kim, Sungsik Kim, and Chong-Kwon Kim. Bezier curve-based path planning for autonomous driving. In *2012 12th International Conference on Control, Automation and Systems (ICCAS)*, pages 1536–1539. IEEE, 2012.

[15] MarianSigler. A cubic Bézier curve with four control points. Wikimedia Commons, 2006. Public Domain. Accessed on September 29, 2025.

[16] Tobias Recker, Mohammad Ehsan Matour, and Annika Raatz. A simple and modular approach to path planning for tractor-trailer robots based on modification of pre-existing trajectories. 2021.

[17] Yan Ding. The Shuffle AI Blog. Shuffle AI, 2024. Accessed on September 29, 2025.

[18] Riccardo Lattarulo, Sante Gaglione, and Francesco Di Rauso. A review of path quality metrics for autonomous vehicle navigation. *Applied Sciences*, 11(16):7591, 2021.

[19] Erann Gat. On three-layer architectures. In David Kortenkamp, R. Peter Bonasso, and Robin Murphy, editors, *Artificial Intelligence and Mobile Robots: Case Studies of Successful Robot Systems*, pages 195–210. MIT Press, 1998.

List of Figures

1 Photo of the rover not included in this version 2

1.1 Simplified planar kinematic structure of the articulated robot. [1] 6
1.2 Representation of the turning radius of curvature of the front module. [1] 9
1.3 The Perception-Planning-Control pipeline with arrows showing the flow of information between modules. 12
1.4 The transformation of a 2D world with a robot and obstacles into a Configuration Space where the robot is a point and obstacles are inflated. 14
1.6 Illustration of the Pure Pursuit controller with a robot, a path, and the lookahead distance to a target point. [8] 17
1.5 The PRM construction process showing: (a) obstacles in a space with start and goal, (b) randomly sampled nodes and valid samples, (c) the constructed roadmap with connected, collision-free nodes, and (d) the final path found by Dijkstra's algorithm. 18

2.1 The MATLAB development environment. The platform's integrated visualization and specialized toolboxes for robotics and navigation provided a powerful framework for algorithm prototyping and validation. 21
2.2 Environment modeling process. From a raw image (a), a binary occupancy map is created (b), which is then inflated (c) to create a conservative planning space with safety buffers around obstacles. 23
2.3 Kinematic model of the articulated rover, showing the key geometric parameters (`Ltr`, `Ltl`, `Wtr`, `Wtl`, `Look` and `Lh`) that govern its motion and articulation. . . . 24
2.4 The PRM path generation process. Randomly sampled nodes (blue) are connected to form a roadmap (green), which is then searched by Dijkstra's algorithm to find an optimal coarse path (red). 25
2.5 The offline path modification framework. This iterative process simulates a naive path, detects hazardous segments, replaces them with safe alternatives, and validates the final result before execution. 27

3.1 A cubic Bézier curve defined by four control points. The curve interpolates the start and end points and is fully contained within the convex hull (dotted line), providing a powerful guarantee for collision avoidance. From [15]. 29

3.2 The Precise Swept-Path Envelope. The green area represents all space occupied by the trailer during the simulation. The overlap with the black obstacle indicates a collision, triggering the hazard detection. 32
3.3 The Two-Factor Hazard Trigger. The maneuver is not flagged as dangerous because the vehicle's bounding circle (yellow) is intersecting the inflated obstacle zone, but the hitch angle is within its threshold, indicating straight navigation in a constrained space. 33
3.4 The Causal Maneuver Window. Instead of only flagging the path segment where the hazardous event occurs (left image), the algorithm shifts the start point backward by a causal distance, capturing the entire maneuver (right image) that led to the event. 34
3.5 Informed Replanning with Bézier Curves. The hazardous nominal segment (magenta) is replaced by an optimized, C^2 continuous Bézier curve (described by points P1 and P2) that is validated against a vehicle costmap to ensure it is collision-free and maintains a safe distance from obstacles. 35

4.1 The geometric principle of the Pure Pursuit controller. The steering command is calculated to place the vehicle on a circular arc that intercepts the reference path at a target point. Illustration by Ding [17] 38
4.2 Definition of Lateral Error, the primary metric for evaluating path tracking accuracy. 39
4.3 A plot showing the reference path and the simulated trajectories for all three controllers attempting to track a straight path. 40
4.4 A plot of Lateral Error vs. Time attempting to track a straight path 41
4.5 A plot showing the reference path and the simulated trajectories for all three controllers on a curved path. The corner-cutting of the Fixed Pure Pursuit, the potentially tighter tracking of the Adaptive Pure Pursuit, and the overshoot of the PID controller are visible in this image. 41
4.6 A plot of Lateral Error vs. Time attempting to track a curved path 42
4.7 A plot showing the reference path and three simulated trajectories for the Fixed Pure Pursuit controller, each with a different lookahead distance. The plot clearly shows the small-lookahead path oscillating and the large-lookahead path cutting the corner. 43
4.8 A plot of Lateral Error vs. Time, showing the error curves for the three different lookahead distances, illustrating the trade-off between oscillation and large, smooth corner-cutting error. 43

5.1 The initial smoothed reference path for the test scenario. While geometrically continuous and collision-free for a point-mass, its safety for the articulated rover is unverified. 45

5.2 Collision detection via the Precise Swept-Path Envelope. The green area shows the trailer's true swept path, with the overlap and red markers indicating an unambiguous collision during the tight turn. 46
5.3 The identified hazardous segment for modification. The Maneuver Window algorithm has correctly framed the entire maneuver leading to the collision. . . 47
5.4 The modified trajectory. The hazardous segment has been replaced by an optimized Bézier curve that provides more clearance. 47
5.5 Successful validation of the modified path. The new paths tracked by truck and trailer confirm the effectiveness of the modification. 48
5.6 Risk detection via the Bounding Circle and Hitch Angle. The purple color highlights points where the vehicle was simultaneously too close to an obstacle and in a state of high articulation, flagging the maneuver as high-risk. 49
5.7 The final modified path generated based on Method B's detection. 50
5.8 Successful validation of the risk-based modification. The new simulation shows no hard collisions, confirming that the hazardous portions of the path have been successfully mitigated by the new path. 51
5.9 Comparative overlay of the final trajectories. The path generated by the proactive Method B (red) takes a wider, more cautious approach than the path from the reactive Method A (blue). 52
5.10 Vehicle clearance throughout the simulation on the path modified by **Method A (Envelope)**. The trailer clearance (red) is the critical factor, touching a minimal value lower than $0.3m$ in the tightest turns. 54
5.11 Vehicle clearance throughout the simulation on the path modified by **Method B (Bounding Circle)**. The proactive modification results in a substantially larger minimum trailer clearance throughout the maneuver. 55